MDPI

Article

A Season-Structured Validation and Governance Architecture for Autonomous Strategy-Proposal Generation in Algorithmic Trading

Pedro Sobreiro [1,4,*], Domingos Martinho [2,3], Pedro Ramos [2,4], António Pratas [2]

[1] Sports Science School of Rio Maior, Polytechnic Institute of Santarém, 2040-413 Rio Maior, Portugal; sobreiro@esdrm.ipsantarem.pt
[2] Instituto Superior de Gestão e Administração de Santarém (ISLA Santarém), Polytechnic University, Rua Dr. Teixeira Guedes, 31, 2000-029 Santarém, Portugal;
[3] Research Centre for Business Sciences (NECE), Estrada do Sineiro 56, 6200-209 Covilhã, Portugal;
[4] Life Quality Research Centre (LQRC), Complexo Andaluz, Apartado 279, 2001-904 Santarém, Portugal
* Correspondence: sobreiro@esdrm.ipsantarem.pt

Abstract

Autonomous trading research pipelines can generate large volumes of machine-produced strategy proposals, but testing them without leakage, overfitting, or silent selection bias remains an open problem. Existing autonomous trading systems primarily optimize strategy generation, whereas considerably less attention has been devoted to the governance of autonomous research pipelines that continuously generate, validate, and discard machine-generated hypotheses under strict temporal-integrity constraints. We introduce a season-structured validation and governance architecture that addresses this gap by combining leakage-free feature constraints, layered temporal validation, Bayesian risk-parameter optimization, and season-structured human oversight into a single reproducible workflow: within each season a local large language model (LLM) generates candidate strategies, and across season boundaries a human reviewer updates objectives and gates, forming a mixed-initiative governance layer. Across twelve adaptive seasons, 34,333 proposals produced 329 accepted strategies; two case studies—a BNB/USDT candidate and a BTC/USDT replication—passed the predefined validation gates and showed positive passive-holdout performance under conservative calendar-time risk metrics, while a controlled ablation study showed no consistent statistical evidence that the LLM-selected feature combination significantly outperforms random subsets drawn from the same catalog. The contribution is therefore the validation and governance architecture itself, which turns a large stream of machine-generated proposals into a small set of credibly testable candidates, rather than any superior LLM feature intuition. We are deliberately conservative: because human-in-the-loop adjustments between seasons consume out-of-sample information, the cross-season design is interpreted as a prolonged model-selection procedure; the reported case studies are therefore descriptive evidence of candidate rejection, not independent prospective proof of future profitability.

Keywords: prescriptive analytics; autonomous analytics pipeline; algorithmic trading; large language models; automated machine learning

Received:
Accepted:
Published:

1. Introduction

Algorithmic trading research is traditionally labor-intensive. A researcher hypothesizes a signal, writes code, engineers features, tunes parameters, and validates the result

out-of-sample, and each step is susceptible to look-ahead bias, data snooping, and overfitting [5,19]. The proliferation of candidate factors has made multiple-testing corrections essential, since even random rules yield spuriously significant backtests at scale [20]. Recent advances in large language models (LLMs) have opened the possibility of automating parts of this pipeline—from code generation [12] to factor mining [43]—prompting dedicated surveys [16].

Most published frameworks still rely on heavy human supervision or treat the LLM as a black-box oracle without explicit safeguards against leakage or overfitting [5,26,38]. At the same time, the emerging generation of autonomous quant-agent systems has focused on capability—richer agent architectures and verifiable task benchmarks [24,50]—while recent studies show a gap between simulated evaluation and deployable trading performance [29,38]. Existing autonomous trading systems primarily optimize strategy generation, whereas considerably less attention has been devoted to the governance of autonomous research pipelines that continuously generate, validate, and discard machine-generated hypotheses under strict temporal-integrity constraints.

The objective of this study is to propose and evaluate a season-structured, human-in-the-loop architecture in which an LLM agent autonomously proposes, codes, validates, and backtests directional cryptocurrency strategies, with leakage prevention and overfitting control as core design constraints. Trading is treated as a closed-loop analytics problem in which predictive modeling, simulation, and optimization prescribe actionable decisions subject to temporal-integrity and statistical constraints [9,37].

This study is guided by three research questions. RQ1: Does a season-structured, human-in-the-loop architecture, through leakage-free feature constraints and layered validation, constrain the risk of leakage and overfitting in autonomous strategy discovery? RQ2: What role do the governance mechanisms embedded at season transitions—acceptance gates and human review—play in the reliability of an autonomous LLM-driven research pipeline? RQ3: Does the LLM contribute predictive or feature-selection value beyond what a constrained random search over the same leakage-free catalog would achieve?

In this framework, an experimental *season* is the operational unit of a human-in-the-loop interaction cycle for quantitative investment, grounded in mixed-initiative governance principles [4,22] and generative curriculum learning [7]. Inside a season, the LLM agent operates autonomously under fixed constraints. At the season boundary, a human reviewer inspects the accepted candidates, updates the objective, tightens acceptance gates, and resets the scoring weights. The sequence of seasons implements a curriculum: earlier seasons use looser gates to explore the strategy space, later seasons stricter temporal-integrity and holdout criteria. Confining unbounded autonomous search to bounded, governed seasons makes leakage prevention and overfitting control enforceable as first-class architectural invariants.

The paper combines three elements. Methodologically, we integrate LLM-driven proposal generation, leakage-free feature constraints, layered validation, and an evolving composite score into a single autonomous research loop—a combination that is novel, to our knowledge, inside a season-structured, human-in-the-loop pipeline. The open-source implementation—a local 7B-parameter LLM, AST-based future-leak detection, and Numba-accelerated backtesting—makes thousands of autonomous iterations feasible on consumer hardware. Empirically, two candidates (BNB/USDT and BTC/USDT) passed the pipeline's gates and showed positive passive-holdout performance under conservative calendar-time metrics (Sections 4.2–4.3).

The LLM works as a high-volume proposal generator, not as a source of superior financial intuition. The ablation study (Section 4.9) finds no consistent statistical evidence that LLM-selected feature combinations outperform random subsets drawn from the same

catalog; the scientific contribution is therefore the validation and governance architecture, which turns a large stream of machine-generated proposals into a small set of testable candidates.

Because the human reviewer updates season objectives, acceptance gates, and scoring weights after observing prior holdout results, the sequence of seasons constitutes an adaptive meta-search—a prolonged model-selection procedure rather than a single pre-registered protocol—that gradually consumes out-of-sample information. The reported passive-holdout results are therefore evidence that the architecture can reject unsound candidates, not prospective proof that the surviving strategies possess durable trading edge. We return to this limitation in Sections 4 and 5.

The paper proceeds in six sections. Section 2 reviews related work; Section 3 details the methodology; Section 4 reports the search across twelve seasons and the two case studies; Section 5 discusses limitations, design principles, and future work; and Section 6 concludes.

2. Related Work

2.1. LLMs for strategy and factor generation

The use of LLMs in quantitative finance has expanded rapidly, prompting dedicated surveys of the area [24]. Kou et al. [27] propose a risk-aware multi-agent system that uses LLMs to generate alpha factors and dynamically weight them. Wang et al. [43] introduce Alpha-GPT, an interactive framework in which an LLM proposes factors that a human quant refines; Alpha-GPT 2.0 [48] later moves toward self-evolving agents that discover alphas with limited human input. Zhang et al. [49] simulate stock trading with LLM-based agents in near-real-world environments. This line has since split into two strands. On *factor mining*, Yu et al. [47] generate synergistic formulaic alpha collections via reinforcement learning, while Zhang and Maringer [55] use a genetic algorithm to improve recurrent reinforcement learning for equity trading. On *multi-agent execution*, Fang et al. [15] learn intention-aware communication for optimal multi-order execution. These works show that LLMs can generate economically meaningful signals, but they typically operate at the factor or sub-decision level and still require human judgment — or a separate learned controller — for execution logic, risk management, and hyper-parameter tuning.

Our work differs in scope: the LLM proposes the *complete* strategy — including entry rules, stop-loss, take-profit, probability threshold, and feature set — and the system validates it end-to-end without human approval of individual candidates.

Chain-of-thought prompting [44] and the tight integration of reasoning with action in ReAct-style agents [46] matter directly for our pipeline. The LLM does not just emit a factor or a signal; it constructs a structured proposal, reflects on previous failures stored in the prompt, and generates executable code that is then validated by the environment. This situates the system between generative AI and autonomous experiment execution.

Factor mining with machine learning is a central topic in empirical asset pricing. Gu et al. [17] show that machine-learning methods can forecast expected returns from large panels of characteristics, while Harvey et al. [20] document how the cross-section of published factors is heavily inflated by data mining. Our pipeline combines both perspectives: it uses gradient-boosted classifiers to mine predictive features, but it constrains the search space with a leakage-free feature catalog and evaluates every candidate on a holdout set that is never used during optimization.

2.2. Automated machine learning for trading

AutoML methods such as genetic programming [2,10], Bayesian optimization [1], and reinforcement learning [42] have long been used for trading strategy discovery. FinRL [29] provides a comprehensive reinforcement-learning framework for quantitative finance,

while LLM-GA [51] combines genetic algorithms with LLM-generated seeds. Sun et al. [39] survey reinforcement-learning methods for quantitative trading. Recent reinforcement-learning work has shifted from raw predictive power to *deployability*: Detzel et al. [14] compare asset-pricing models after transaction costs, and DeepAries [53] learns *when* to rebalance rather than just *what* to hold. These approaches can search large strategy spaces, but they often lack interpretability and require careful reward shaping to avoid overfitting.

General-purpose AutoML frameworks such as TPOT [56], AutoGluon [57], and Auto-sklearn [58] automate model selection and hyper-parameter tuning for tabular tasks, but they are not designed for financial time series. They lack native temporal cross-validation, do not enforce embargo or purging between folds, and do not incorporate trading-specific constraints such as slippage, fees, or per-bar lookahead checks. Trading therefore requires an AutoML architecture in which leakage prevention and out-of-sample validation are first-class components rather than optional wrappers.

Our system uses Bayesian optimization (Optuna) only for the final risk-management parameters of a strategy whose predictive features have already been selected by the LLM. This separation of feature-design from parameter-tuning reduces the effective search space and improves interpretability. Here the staged search mirrors the automated machine-learning literature, where model structure and hyper-parameters are optimized sequentially so the search remains tractable [23].

2.3. Overfitting, multiple testing, and statistical rigor

Backtest overfitting is the main challenge in algorithmic research. Bailey and López de Prado [5] show that the maximum Sharpe ratio observed after many independent trials is biased upward, and they propose the Deflated Sharpe Ratio (DSR) to correct for selection bias and non-normality. Harvey and Liu [19] emphasize the importance of out-of-sample testing and multiple-testing adjustments in finance. Lo [30] discusses the statistical properties of Sharpe ratios under non-i.i.d. returns. For dependent return series, block-bootstrap methods such as the stationary bootstrap [59] and the broader theory of resampling for dependent data [60] are preferred over naive i.i.d. resampling because they preserve serial dependence; the bootstrap diagnostics reported here use naive resampling and should therefore be read as descriptive lower bounds on uncertainty. Beyond this foundational work, Jobson and Korkie [25] and Ledoit and Wolf [28] developed robust inference procedures for Sharpe ratios that remain valid under non-normality and dependence, while Romano and Wolf [32] provided stepwise multiple-testing corrections that control the family-wise error rate when many strategies are examined in sequence. Sullivan, Timmermann, and White [33,34] and White's [35] reality check showed that apparently profitable trading rules often disappear once data-snooping is accounted for, and Hansen [36] developed a test for superior predictive ability that is directly applicable when a large panel of candidate strategies is screened. Harvey et al. [20] survey the cross-section of published factors and show that most apparent anomalies are likely artifacts of data snooping. These concerns have acquired new urgency in the age of automated discovery: a recent post-mortem of one highly automated, self-evolving trading system documents catastrophic divergence between validation and live performance [52], recent work argues that statistical rigor must be enforced *structurally* within the discovery loop rather than checked afterward, supported by classical false-discovery-rate control [8], and studies of LLM-based trading systems show that seemingly minor execution assumptions can render reported results irreproducible [54].

We put these insights into practice: every reported strategy is evaluated on a holdout set that was never used by Optuna, and we compute bootstrap p-values and a DSR approximation to estimate how likely the observed performance is to have arisen by chance.

Following [8] and the reproducibility audit of [54], these safeguards are enforced *inside* the autonomous loop — through leakage-free feature constraints and layered temporal validation — rather than applied only to the final selected strategy.

2.4. Composite scoring and strategy selection

Strategy selection in automated research calls for more than maximizing a single in-sample metric. While standard quantitative workflows evaluate strategies across multiple, independently reported dimensions—including the Sharpe ratio, drawdown, win rate, and turnover—as distinct diagnostic statistics [31], automated discovery engines often combine these metrics into a single composite fitness function to guide heuristic search [2,51]. Recent work has proposed formal statistical adjustments to penalize selection bias and multiple-testing inflation, such as the Haircut Sharpe Ratio [19] and the Deflated Sharpe Ratio [5], as well as combinatorial validation frameworks to stress-test regime instability [7,31]. Yu et al. [47] evaluate formulaic alphas on several efficiency-aware axes rather than by return alone.

Our composite score combines validation Sharpe, holdout Sharpe, holdout return, maximum drawdown, and a cross-validation AUC penalty. The score is reweighted across seasons based on observed failure modes, which creates a curriculum for the LLM agent.

2.5. Prescriptive analytics and autonomous analytics pipelines

Predictive analytics forecasts future outcomes from historical data, while prescriptive analytics recommends or automates decisions that optimize an objective under constraints [13,37]. Bertsimas and Kallus [9] formalize the shift from prediction to prescription, showing how machine-learning predictions can be combined with optimization to produce actionable policies. Separately, a strand of *autonomous analytics pipelines* has appeared in quantitative finance, in which agents iteratively propose, backtest, and refine strategies: FinAgent [50], for example, formalizes a self-improving loop for quantitative investment. This has prompted verifiable evaluation environments and closed-loop benchmarks for such agents [40,41], reflecting the growing recognition that autonomous quant systems must be judged on reproducible, cost-aware, out-of-sample protocols rather than single headline backtests.

Our contribution is an *offline prescriptive-analytics research pipeline*: it discovers, validates, and selects decision policies, but it does not trade live. Deploying a selected policy would require a separate online system that reads current market features and prescribes positions. We treat the offline research phase as prescriptive analytics because its output is a complete decision policy optimized end-to-end for a stated objective and validated on holdout data; but we do not claim live deployment results. This aligns with the design-science view that a research artifact can be evaluated by its ability to produce well-validated prescriptions even before operational deployment [21]. Relative to the benchmark-driven agent literature [40,41], our emphasis is complementary: we do not focus on richer agents or broader task suites; instead, we focus on a season-structured governance layer that decides which autonomously generated candidates are trustworthy enough to retain.

2.6. Human-in-the-loop research and mixed-initiative control

Fully autonomous research raises questions about control and interpretability. Horvitz [22] introduced the principles of mixed-initiative user interfaces, where humans and intelligent systems collaborate rather than work in isolation. Amershi et al. [4] turned these ideas into guidelines for human-AI interaction, noting that systems should communicate uncertainty, allow human override at appropriate moments, and improve from user feedback over time.

Our pipeline puts these principles into practice by confining full autonomy within a season and keeping human review for season transitions. The human reviewer updates the season objective, acceptance gates, and scoring weights based on the distribution of accepted strategies. This is a mixed-initiative design: the machine runs thousands of experiments and surfaces patterns, while the human steers the research direction and guards against objective drift. In this sense, the season boundary operationalizes mixed-initiative control: it gives the human reviewer explicit authority over objectives and acceptance criteria without requiring case-by-case approval of individual candidates. Table 1 summarizes how the systems discussed in this section compare along the dimensions most relevant to this work.

Table 1. Comparison of autonomous and semi-autonomous trading research systems

System	Focus	Human oversight	Leakage control	Governance layer
LLM-GA [51]	Evolutionary strategy generation with LLM-guided seeds	Not specified	Composite-fitness search; no explicit leakage audit reported	None reported
FinAgent [50]	Multimodal foundation agent with a self-improving loop for quantitative investment	Limited (self-improving loop)	Benchmark-based evaluation	None reported
FinRL [29]	Deep reinforcement-learning framework for automated trading	User-configured	User-defined environment and reward shaping	None reported
Alpha-GPT [43]	Interactive LLM alpha mining	Human quant refines each proposed factor	Not specified	Human-in-the-loop at factor level
This work	Complete strategy proposals by a local LLM, validated end-to-end	Human review only at season transitions (15–30 min per transition)	Leakage-free feature catalog, AST-based future-leak detection, layered temporal validation, passive holdout	Season-structured acceptance gates, transition review protocol, and audit trail

3. Methodology

3.1. System overview

Definition 1 (Experimental season)**.** *An* experimental season *is the operational unit of a human-in-the-loop interaction cycle for quantitative investment: a bounded period of autonomous search during which the LLM agent operates under fixed constraints — a fixed objective, acceptance gates, scoring weights, and feature catalog — while all human intervention (inspection of accepted candidates, objective updates, gate adjustments, and score reweighting) is deferred to the season boundary, a review that typically takes 15–30 minutes per transition. The sequence of seasons*

implements a curriculum: earlier seasons use looser gates to explore the strategy space, later seasons stricter temporal-integrity and holdout criteria.

The autonomous research loop has five stages:

1. **Hypothesis generation**: the LLM receives a prompt describing the current season's goal, past failures, and the feature catalog, and proposes a strategy as a JSON object.
2. **Code generation**: the LLM writes Python code implementing the strategy, including feature engineering, model training, and signal generation.
3. **Validation**: a five-layer validator checks syntax, execution, AST-based future-leak detection, required-field presence, and feature-catalog compliance.
4. **Optimization**: Optuna tunes stop-loss, take-profit, and probability threshold on the training/validation window.
5. **Evaluation**: the strategy is simulated on the holdout window; if it passes the acceptance gates, it is recorded as accepted.

The loop repeats until a season budget is exhausted or a target number of accepted strategies is reached. Accepted strategies are ranked by a deploy score, and the best is selected as the production candidate. Figure 1 shows the overall architecture, and Figure 2 summarizes the season-level timeline: each season runs autonomously under fixed gates and objectives and ends in a human review that gates the transition to the next season, culminating in the final candidate selection.

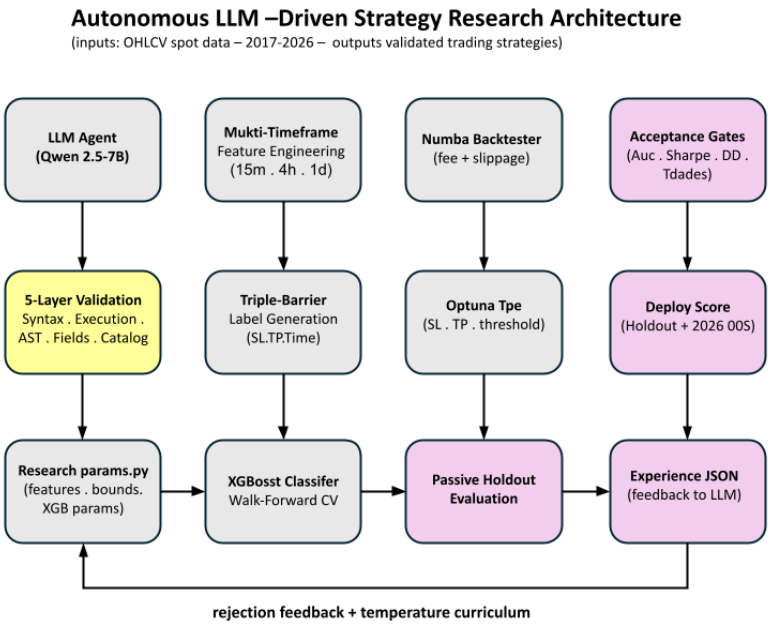

Figure 1. Autonomous strategy research architecture incorporating LLM-based proposal generation.

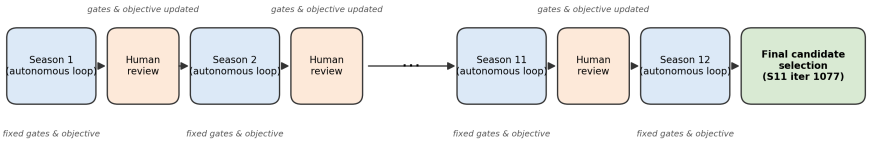

Figure 2. Season timeline: each season runs as an autonomous loop under fixed acceptance gates and objectives; human review occurs only at season transitions, where gates and objectives are updated, and the sequence ends in the final candidate selection.

3.2. Feature catalog and temporal-ordering constraints

All features are computed from OHLCV data at 15-minute, 4-hour, and daily timeframes. The catalog includes technical indicators (RSI, MACD, Bollinger Bands, ADX, ATR, Stochastic RSI, EMA differences), volatility measures, and lagged returns. Every feature is normalized by price or bounded to the current bar, and higher-timeframe indicators are merged into the 15-minute bars only after the corresponding candle has closed. The merge module shifts each higher-timeframe timestamp from its open time to its close time and then uses a backward-looking as-of merge; a daily value observed on day t therefore first becomes available to the 15-minute model at the close of day t. The validator parses the generated code into an AST and rejects any strategy that references absolute prices after the decision point or that accesses data outside the current bar. These precautions are consistent with standard defenses against leakage in data mining [26] and backtest overfitting [6], and with López de Prado's [31] emphasis on purging and embargo in financial cross-validation: observations that overlap a test period due to long investment horizons, or strongly autocorrelated returns, must be excluded from training and from immediate post-test retraining. Our one-bar look-ahead lag for higher-timeframe merges and the AST-based price-reference check implement these principles in code. We treat this temporal-integrity layer as a design invariant; it is enforced in code review and verified by the out-of-sample stability reported in the results.

3.3. Training and backtesting engine

Labels are generated with a triple-barrier method [31]: each trade has a profit-taking barrier, a stop-loss barrier, and a vertical time barrier. An XGBoost classifier [11] is trained on the training set and predictions are converted into signals. The backtester simulates trades using Numba-accelerated code. Costs are modeled as a 0.20 % round-trip fee plus 0.10 % slippage on entry, following the transaction-cost framework used in optimal execution literature [3]. Stop-loss and take-profit exits are executed at the triggered level without additional slippage in the baseline configuration; Section 4.5 reports a stress-test that adds up to 0.30 % symmetric slippage on all exits.

All Sharpe ratios reported by the engine and in the statistical audit are computed from daily equity changes and annualized with $\sqrt{365}$, the standard convention for cryptocurrency markets. This convention is used consistently across validation, holdout, and the secondary selection holdout periods.

An optional volatility kill-switch (`atr_kill`) zeros model probabilities when the 15-minute ATR exceeds a rolling multiple, preventing entries during extreme volatility spikes. The ATR and ADX indicators used for volatility and trend-strength filtering follow the original definitions of Wilder [45]. In the selected production candidate (S11 iter 1077), the kill-switch was configured at 3.0 rolling standard deviations. Candidate acceptance also requires a maximum drawdown below -15 %.

To mitigate overfitting, we use a nested temporal split:

- **Training**: oldest 60 % of the in-sample window.
- **Validation**: next 20 %, used for early stopping and Optuna.
- **Holdout**: most recent 20 %, never seen by Optuna.

For Season 12, the split was extended to 2017–2020 training, 2021–2023 validation, and 2024–2025 holdout.

3.3.1. Walk-forward cross-validation

Inside the training window, XGBoost hyper-parameter search and the reported cross-validation AUC use scikit-learn's `TimeSeriesSplit` with three and five folds respectively. Each fold is a contiguous block: fold t is trained on all observations up to a split point

and tested on the immediately following block. This preserves temporal ordering but does not apply explicit purging or embargo between folds; because labels are generated with a triple-barrier that closes trades at a fixed horizon or barrier, the overlap between adjacent training and test folds is limited. The final model is retrained on the full training window (the years between `train_start` and `train_end`, not the validation or holdout windows) after hyper-parameter selection via walk-forward cross-validation; the validation and holdout windows are never used for model fitting, only for evaluation and deploy scoring.

3.4. LLM agent and prompt engineering

The LLM agent is a local Qwen2.5-7B-Instruct model served via llama.cpp. The choice of a 7B-parameter model is deliberate: it is small enough to run locally on consumer-grade GPUs via llama.cpp, which keeps the research loop private (no strategy code or market data is sent to external APIs) and eliminates per-token API costs across tens of thousands of iterations. Despite its size, the model generates syntactically valid Python strategy proposals in the constrained JSON schema used by the pipeline.

Across Season 11, the five-layer validation and scoring infrastructure rejected 94.4% of the 3,797 candidates: 18.5% failed code-validation (syntax, execution, AST future-leak, or feature-catalog compliance), 9.1% were duplicate parameter configurations, 18.8% converged to a previously discovered Optuna optimum, and 42.7% passed all code and optimization layers but failed the performance gates (AUC, validation Sharpe, holdout Sharpe, or drawdown). Only 5.6% were accepted. These rates indicate that the bottleneck is not the LLM's proposal quality per se, but the multi-layer validation and scoring layers that filter out unsound candidates.

The prompt contains:

- The season objective (e.g., "improve holdout Sharpe on BTC").
- A summary of the previous season's accepted strategies and common failure modes.
- The feature catalog with exact function signatures.
- A JSON schema for the strategy proposal.
- Examples of valid and invalid code patterns.

The agent keeps a memory of recent failures to avoid repetitive proposals. After each season, a human-in-the-loop review updates the season objective and acceptance gates based on the distribution of accepted strategies. This is the primary point of human supervision; within a season, the loop runs without human approval of individual candidates. This mixed-initiative design follows the principles of Horvitz [22] and Amershi et al. [4]: the system performs the bulk of routine experimentation and surfaces structured results, while the human retains control over high-level goals and value alignment. The criteria used at each major transition are documented in Supplementary Table S2.

3.4.1. Governance and accountability

Although the loop inside a season is autonomous, several mechanisms constrain the scope of autonomous decisions and allocate accountability.

Prompt and schema constraints. The LLM cannot propose arbitrary code. It must emit a JSON object with fixed fields (`features`, `entry_logic`, `sl_pct`, `tp_pct`, `threshold`, `adx_min`, `atr_kill_threshold`, `xgb_params`) and use only functions from the leakage-free feature catalog. The prompt includes the season objective, a summary of the last 10–20 rejected proposals, and three in-context examples of valid and invalid code patterns. The temperature is initialized at 0.7 and annealed downward when the acceptance rate stalls, reducing exploration once the search has found a productive region.

Definition 2 (Season transition)**.** *A* season transition *is the human-governed boundary between two consecutive seasons, at which the season-transition review protocol is applied. At each transition the human reviewer inspects: (i) the distribution of accepted strategies by AUC, Sharpe, and drawdown; (ii) the most common rejection reasons; (iii) the feature-importance patterns of accepted models; and (iv) any evidence of objective drift (e.g., falling validation Sharpe despite rising holdout Sharpe). Based on this, the reviewer updates the season objective, adjusts one or two acceptance bounds, and reweights the composite score. The reviewer records the rationale in* `season_notes.md`*, which becomes part of the audit trail.*

Accountability allocation. We adopt a simple RACI-style model for the offline research phase:

- **Responsible (autonomous agent):** proposing candidates, running backtests, and logging results.
- **Accountable (human researcher):** setting season objectives, acceptance gates, and scoring weights; interpreting results; and deciding whether a candidate is promoted to further study.
- **Consulted (code validator, statistical audit):** enforcing hard constraints and flagging suspicious patterns.
- **Informed (future deployment system):** receiving the selected candidate and parameters, but not involved in research decisions.

This model applies to the offline research pipeline only. A subsequent live system would require a separate governance layer with real-time monitoring, circuit breakers, and compliance sign-off that are outside the scope of this paper. This operational structure instantiates, at the level of daily practice, the mixed-initiative governance principles [4,22] and the design-science view of a validated research artifact [21] introduced in Section 1.

3.5. Acceptance gates and composite score

Definition 3 (Acceptance gate)**.** *An* acceptance gate *is a configurable, season-specific threshold that a candidate strategy must satisfy to be recorded as accepted. A strategy is accepted only if it satisfies all of the following configurable gates, which changed across seasons:*

- *Cross-validation AUC above a season-specific threshold (typically $\geq$ 0.52–0.55).*
- *Sharpe ratio on the validation window above a season-specific threshold (typically $\geq$ 0.5–1.0).*
- *Sharpe ratio on the passive-holdout window above a season-specific threshold (typically $\geq$ 0.0–0.5).*
- *Maximum drawdown within the backtest objective bounds (the objective function rejects configurations with excessive drawdown).*
- *At least a minimum number of trades in the validation window.*
- *All code-validation layers passed.*

Strategies are ranked by a composite score that combines the validation Sharpe ratio, the passive-holdout Sharpe ratio, the holdout return, the maximum drawdown, and a cross-validation AUC penalty. Formally, the ranking objective can be written as:

$$S = w_1 \cdot \text{Sharpe}_{\text{val}} + w_2 \cdot \text{Sharpe}_{\text{hold}} + w_3 \cdot \text{Ret}_{\text{oos}} - w_4 \cdot |\text{DD}_{\text{max}}| - w_5 \cdot \max(0, 0.5 - \text{AUC}_{\text{cv}})$$

where the weights w_i were adjusted across seasons. The AUC term penalizes only below-chance discriminative ability; it does not penalize AUC values above 0.5, because higher validation AUC is desirable once the leakage checks have passed. In Seasons 9–12, the weights emphasized holdout Sharpe and drawdown control; strategies with negative

holdout Sharpe or validation-holdout decay were heavily penalized. Inside the Bayesian optimizer, the objective function changed from maximizing the minimum validation Sharpe across sub-windows (score mode) to maximizing the average holdout return (profit mode), with hard constraints on drawdown and trade count.

3.6. Deployment score and model selection

For deployment selection, we use a deploy score that averages the passive-holdout Sharpe and the Sharpe on the secondary selection holdout (January–March 2026 for S11):

$$\text{Deploy\_score} = \frac{\text{Sharpe}_{\text{hold}} + \text{Sharpe}_{2026}}{2}$$

Because the deploy score includes Sharpe_{2026}, the 2026 period works as a **secondary selection holdout**: it was used to choose the production candidate from the 30 top holdout candidates that were re-evaluated for deployment from Season 11. It is therefore not an independent prospective test and should be treated only as a noisy post-selection observation. The selected candidate, S11 iter 1077, had the highest deploy score among these 30 re-evaluated candidates; that score is driven primarily by the passive-holdout Sharpe because the 2026 window contains only four trades and its Sharpe is extremely noisy.

The selected candidate's passive-holdout Sharpe (2.58) comes from a population of 212 accepted strategies in Season 11, so it is also subject to selection bias. Under the simplifying assumption that the 212 accepted holdout Sharpes are i.i.d. normal with the observed mean (1.37) and standard deviation (0.61), the expected maximum among 212 draws is approximately 3.06 (95 % interval: 2.66–3.62), and the observed maximum holdout Sharpe in that population (2.95, with the selected candidate at 2.58) lies within the range expected from pure selection; however, the positive mean of the accepted strategies (1.37) indicates that the accepted population as a whole has positive holdout performance. This selection-bias calculation follows the Extreme Value Theory framework developed by [5] to estimate performance inflation under multiple testing.

The Q1 2026 Sharpe is based on only four trades and is consequently very noisy; it should not be interpreted as precise evidence of live edge, nor used to rank the candidate against other candidates. We report it as a preliminary post-selection observation rather than as an independent out-of-sample result.

3.7. Experimental configuration and computational budget

Table 2 summarizes the two strategies analyzed in detail. Table 3 reports how the search evolved across seasons. The full feature catalog is provided in Supplementary Table S1, and the inter-season design decisions are documented in Supplementary Table S2.

Table 2. Case-study strategies

Feature	S11 iter 1077 (production)	S12 iter 5502 (BTC replication)
Asset	BNB/USDT	BTC/USDT
Training period	2022–2023	2017–2020
Validation period	2022–2024 (walk-forward)	2021–2023
Holdout period	2025	2024–2025
Secondary selection holdout	Jan–Mar 2026	—
Validation AUC	0.5721	0.5321
Validation Sharpe	1.69	1.39
Holdout Sharpe	2.58	1.15

Feature	S11 iter 1077 (production)	S12 iter 5502 (BTC replication)
Total holdout return	19.1 % (2025)	27.7 % (2024–2025, compounded)
Maximum drawdown (validation)	-8.89 %	-8.13 %
Win rate (final validation year)	81.2 %	90.9 %
Number of trades (validation)	182 (2022–2024)	38 (2021–2023)
Stop-loss / Take-profit	7.85 % / 7.07 %	9.70 % / 9.12 %
Probability threshold	0.857	0.890
Regime filter	ADX $\geq$ 22, ATR kill 3σ	ADX implicit in features
Modeled costs	0.20 % fee + 0.10 % slippage	0.20 % fee + 0.10 % slippage

Note: All metrics in this table are taken from the pipeline backtest recorded in `best_models/season_N/` *the canonical source for the reported results. Sharpe ratios are computed by the backtest engine from daily equity changes over all calendar days (including days with no closed position) and annualized with* $\sqrt{365}$*; validation and holdout Sharpes are simple averages of the per-year Sharpes within each period. The equity curve starts at EUR 500 and compounds chronologically within the validation years and, independently, within the holdout years; the total holdout return is the compounded return over the holdout years alone (*`retorno_total_oos_pct`*). Maximum drawdown, number of trades, and win rate refer to the validation period, as recorded by the pipeline. The holdout period was never used during hyper-parameter optimization; the Jan–Mar 2026 period was used to select the production model from 30 re-evaluated candidates and is therefore a noisy post-selection observation (four trades), not an independent prospective test.*

The experiments were run on a local workstation with a consumer GPU serving the Qwen2.5-7B-Instruct model via llama.cpp. We did not log exact wall-clock time per iteration, but the full study (34,333 iterations across twelve seasons) required about 6–8 weeks of continuous execution, including restarts after memory-related crashes in early seasons. The LLM inference and Optuna optimization together consumed an estimated 500–800 kWh. Per-iteration latency varied from seconds (for strategies rejected at the code-validation layer) to several minutes (for strategies that ran the full Optuna loop). Future work will add instrumentation to record precise GPU hours and energy consumption.

3.8. Reproducibility protocol

To enable independent replication, we fixed the software environment, random seeds, data sources, and execution flow described below. All code, experiment logs, and generated artifacts are available in the repository associated with this submission.

Persistent archive. All code, experiment logs, and generated artifacts are publicly available at https://github.com/pesobreiro/algo-autoresearch-paper (MIT license) and are permanently archived at Zenodo (DOI: https://doi.org/10.5281/zenodo.21160587).

Software environment. The pipeline was executed under Python 3.13.11 with the following core package versions: pandas 3.0.1, NumPy 2.4.3, Numba 0.65.1, XGBoost 3.3.0, scikit-learn 1.9.0, Optuna 4.9.0, Joblib 1.5.3, PyArrow 12.0 or later, and llama.cpp serving Qwen2.5-7B-Instruct (quantized GGUF, Q4_K_M). The multi-timeframe feature-merge module is imported from `features.technical` (commit hash recorded in `requirements.txt`). The full dependency list is in `requirements.txt`. We recommend creating a dedicated virtual environment and installing the pinned versions; newer versions may alter random

number generation or sampler behavior and are not guaranteed to reproduce the exact numeric results reported here.

Random seeds. Determinism was enforced at three points: (i) the XGBoost classifier uses `random_state=42`; (ii) Optuna's TPE sampler is initialized with `seed=42` and `n_startup_trials=20` for the SL/TP/threshold study (and `n_startup_trials=max(5, n_trials // 4)` when XGBoost hyper-parameters are also tuned); (iii) scikit-learn's `TimeSeriesSplit` is used for walk-forward cross-validation. Because some operations rely on Numba JIT compilation and BLAS threading, byte-identical reproduction across operating systems is not guaranteed; the statistical conclusions, however, are robust to the residual non-determinism.

Hardware. The main experiments ran on a Linux workstation with an NVIDIA GeForce RTX 2060 (6 GB) and an x86_64 CPU. The LLM server (`llama-server`) offloaded 32 layers to the GPU; the remaining CPU layers introduced per-token latency but no change to the generated distribution beyond temperature effects. CPU-only execution is possible but substantially slower.

Data. OHLCV bars at 15-minute, 4-hour, and daily resolutions were downloaded from the Binance public API and stored as Parquet files in a dedicated data directory (`~/crypto_data`). The raw data are publicly available; the ticker mappings are `bnb_15m_usdt_binance`, `bnb_4h_usdt_binance`, `bnb_1d_usdt_binance` for BNB/USDT and `btc_15m_usdt_binance`, etc., for BTC/USDT. We used the closing prices from the daily series to compute the buy-and-hold benchmarks. A `download_data.py` helper script in the repository performs the exact Binance API calls used in this study and records the date ranges of each file.

Repository structure. The repository contains:

- `autoresearch/` — LLM client, validator, tracker, and runner.
- `pipeline/` — label generation, XGBoost training, Optuna backtest, feature catalog, and `research_params.py` (the single file the LLM is allowed to rewrite each iteration).
- `experiments_s{season}/` — one JSON file per iteration with metrics, parameters, and status.
- `best_models/season_{season}/` — accepted model artifacts.
- `deployment/` — scripts to re-run the selected case studies and produce annual performance tables and benchmark comparisons.

Execution flow. A single season is launched with `python main.py` after copying `config.yaml.example` to `config.yaml` and starting the local LLM server. The runner executes the loop described in Section 3.1: proposal, validation, training, Optuna optimization, and holdout evaluation. The configuration file specifies the ticker, exchange, data directory, season identifier, and acceptance minima; the exact feature sets, indicator parameters, SL/TP/threshold ranges, and XGBoost hyper-parameters are written by the LLM into `pipeline/research_params.py` each iteration. For post-hoc verification, the deployment scripts `deployment/evaluate_models.py` and `deployment/backtest_deploy.py` replay the selected iterations using the saved model and parameters, without calling the LLM.

Reproduction checklist. To replicate the two case studies reported in Section 4: (1) install the pinned environment; (2) download the Binance OHLCV data for BNB/USDT and BTC/USDT at the three timeframes; (3) start the LLM server with the Qwen2.5-7B-Instruct GGUF model; (4) run `python main.py` with the appropriate season number; (5) for the selected iterations, run the deployment scripts with the parameters recorded in `best_models/season_{N}/iter_{XXXX}/meta.json`. Because the search is expensive, a partial reproduction can focus on rerunning the saved case studies, which is computationally cheap and fully deterministic given the saved model and parameters.

4. Results

4.1. Search convergence across experimental seasons

Having characterized the population of 329 accepted strategies drawn from 34,333 proposals across twelve seasons (Table 3), of which the 313 strategies accepted in the four triple-gate seasons (S9–S12) record a comparable passive-holdout Sharpe with mean 1.09 and standard deviation 0.68 (median 0.92; overall acceptance rate 0.96 %), we now examine in detail the two strategies retained as final candidates.

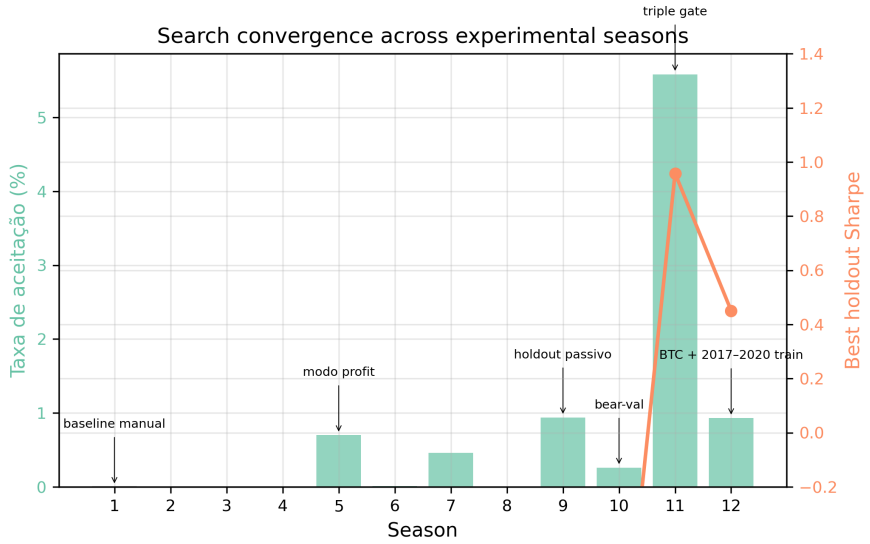

Figure 3. Search convergence across experimental seasons (acceptance rate and best holdout Sharpe).

Table 3. Evolution of acceptance rate and best holdout Sharpe by season

Season	Asset	Iterations	Accepted	Acceptance rate (%)	Best holdout Sharpe	Iteration of best holdout
1	BNB	32	0	0.00	—	—
2	BNB	20	0	0.00	—	—
3	BNB	367	0	0.00	—	—
4	BNB	2,903	0	0.00	—	—
5	BNB	1,419	10	0.70	—	—
6	BNB	12,864	1	0.01	—	—
7	ETH	1,096	5	0.46	—	—
8	BTC	176	0	0.00	—	—
9	BNB	534	5	0.94	1.00	178
10	BNB	1,168	3	0.26	2.41	326
11	BNB	3,797	212	5.58	2.95	1,133
12	BTC	9,957	93	0.93	1.23	1,736

Note: Season 1 was a manual initialization/baseline phase before the automated loop started at Season 2. Scores in S5–S7 used a different scale from S9–S12; the table therefore reports holdout Sharpe as the cross-season comparable metric.

Figure 3 shows the acceptance rate and the best holdout Sharpe ratio per season. Season 1 was the manual initialization baseline (32 iterations, score 0.5607, no formal

acceptance gates); the automated loop started at Season 2. The early automated seasons (S2–S4) produced no accepted strategies because the acceptance gates were too loose or the search space too unconstrained. Season 5 introduced a profit-mode objective and achieved the first acceptances (0.70 %). Season 6 introduced stricter criteria, dropping the acceptance rate to 0.01 % (1 in 12,864). Seasons 9–12 improve steadily in holdout Sharpe, driven by three changes: passive holdout (S9), bear-market validation (S10), and the triple-gate protocol (S11). Season 12 extended the training window back to 2017 and switched to BTC, a test of cross-crypto replication.

The acceptance rate reached its peak of 5.58 % in Season 11, but this reflects a richer proposal distribution, not weaker gates: the average composite score fell from 2.03 in S9 to 1.09 in S11 because the scoring weights were renormalized when the acceptance gates tightened (S9 allowed only five strategies with high validation Sharpe but negative average holdout Sharpe; S11 demanded positive holdout Sharpe and lower drawdown, which compressed the score scale). The best holdout Sharpe rose from 1.00 to 2.95 over the same period, so quality improved even as the raw score range shifted.

4.2. Production model performance

The selected production candidate, S11 iter 1077, traded BNB/USDT over 2022–2025 (validation 2022–2024, passive holdout 2025); the January–March 2026 window was used only for deployment selection and is reported as a post-selection observation. Figure 4 shows the equity curve. Table 4 gives the annual performance. The candidate produced positive returns in every year of 2022–2025 on the validation and passive-holdout windows. The maximum drawdown on the validation period was -8.89 %, well below the -15 % gate. Table 5 compares these returns with a passive buy-and-hold investment in BNB/USDT over the same period.

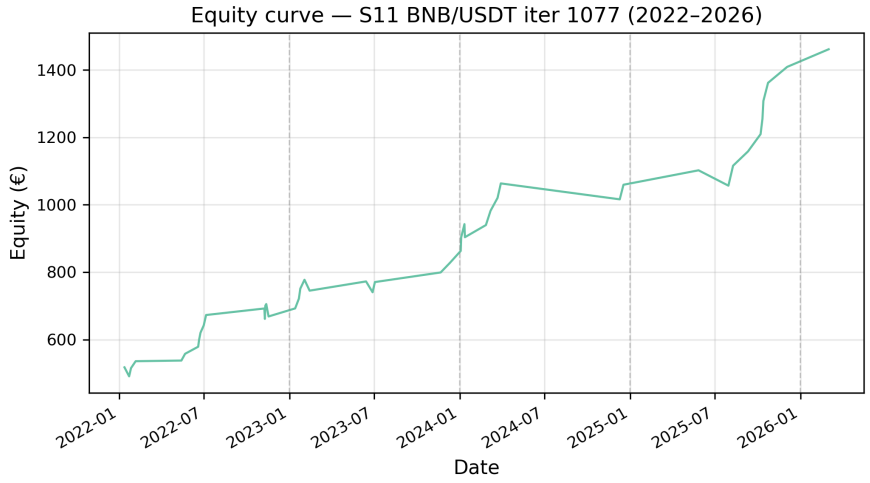

Figure 4. Equity curve — S11 BNB/USDT iter 1077 (2022–2026). The curve spans training (2022–2023), validation (2022–2024), the true holdout year (2025), and a post-selection observation window (Q1 2026, four trades); see Table 2 for exact period boundaries.

The strategy's AUC of 0.5721 shows only marginal predictive power at the individual-bar level; the economic value comes from combining a calibrated probability threshold, asymmetric risk management (7.85 % stop-loss vs. 7.07 % take-profit), and an ADX regime filter [45] that avoids weak-trend entries. The wider stop-loss relative to take-profit came from Optuna optimization under the bear-market validation introduced in Season 10: it gives profitable trades room to survive short-term reversals while the high-probability

threshold and ADX filter keep entry quality high enough that smaller, more frequent gains offset occasional larger losses.

This holdout Sharpe is drawn from a population of 212 strategies accepted in Season 11; Section 3.6 shows via an Extreme Value Theory analysis that the observed value (2.58, against a population maximum of 2.95) falls within the range expected from pure selection, while the accepted population as a whole has positive holdout performance (mean Sharpe 1.37).

Benchmark comparison. Table 5 compares the strategy's annual returns with a passive buy-and-hold investment in BNB/USDT.

Table 4. Annual performance of the selected production candidate (S11 iter 1077, BNB/USDT)

Year	Return (%)	Equity (EUR)	Period
2022	19.59	597.97	Validation
2023	12.95	675.42	Validation
2024	14.53	773.53	Validation
2025	19.10	595.52	Holdout
Compounded (validation 2022–2024)	**54.71**	—	—

Note: Annual returns are derived from the equity curve recorded in `best_models/season_11/iter_1` *(*`equity_500_por_ano`*), the canonical source for the reported results. The equity starts at EUR 500 and compounds chronologically within the validation years (2022–2024) and, independently, within the holdout year (2025); the holdout return of 19.10 % is the pipeline's reported out-of-sample return (*`retorno_total_oos_pct`*). Per-year Sharpe, Sortino, drawdown, trade counts, and win rates are not recorded in* `meta.json` *and are therefore not reported; the recorded aggregates are a validation Sharpe of 1.69, a holdout Sharpe of 2.58, a maximum drawdown of -8.89 % (validation), and 182 trades over the validation years. The Jan–Mar 2026 window is not covered by* `meta.json`*; it was used only for deployment selection and is reported elsewhere as a post-selection observation. Modeled costs: 0.20 % fee + 0.10 % slippage.*

Table 5. Benchmark comparison: S11 iter 1077 vs. BNB/USDT buy-and-hold

Year	S11 iter 1077 (%)	BNB/USDT buy-and-hold (%)	Difference (%)
2022	19.59	-53.29	+72.88
2023	12.95	27.58	-14.63
2024	14.53	124.02	-109.49
2025	19.10	22.10	-3.00
Compounded (validation 2022–2024)	**54.71**	**33.50**	**+21.21**
Holdout (2025)	**19.10**	**22.10**	**-3.00**
Max drawdown (validation)	**-8.89**	**-62.90**	**+54.01**

Note: S11 iter 1077 returns are derived from the equity curve recorded in `best_models/season_11/it` *with modeled costs 0.20 % fee + 0.10 % slippage; equity compounds within the validation years (2022–2024) and independently within the holdout year (2025). BNB/USDT buy-and-hold returns are computed from daily close prices downloaded from the Binance public API; they ignore funding,*

staking, and custody costs. The compounded buy-and-hold figure for 2022–2024 applies the same compounding convention to the annual benchmark returns. The Jan–Mar 2026 window is not covered by `meta.json` *and is omitted.*

4.3. Cross-crypto replication

To test whether the design transfers to a different cryptocurrency, we ran Season 12 on BTC/USDT with a training window starting in 2017. The selected candidate, S12 iter 5502, passed the validation gates and showed a validation Sharpe of 1.39 and a holdout Sharpe of 1.15, with a 27.7 % compounded holdout return (2024–2025) and a maximum drawdown of -8.13 % on the validation period (Figure 5). Table 6 breaks down the annual performance. Returns are positive in every year; the weakest years were 2025 (+10.69 %, a mildly negative year for the BTC benchmark) and 2021 (+10.91 %, a late-bull period). The lower holdout Sharpe relative to S11 reflects Bitcoin's different volatility regime, but the candidate remained profitable and within risk limits.

Table 6. Annual performance of the cross-crypto replication (S12 iter 5502, BTC/USDT)

Year	Return (%)	Equity (EUR)	Period
2021	10.91	554.57	Validation
2022	10.99	615.52	Validation
2023	21.83	749.90	Validation
2024	15.33	576.66	Holdout
2025	10.69	638.33	Holdout
Compounded (validation 2021–2023)	**49.98**	—	—
Compounded (holdout 2024–2025)	**27.67**	—	—

Note: Annual returns are derived from the equity curve recorded in `best_models/season_12/iter_5` *(*`equity_500_por_ano`*), the canonical source for the reported results. The equity starts at EUR 500 and compounds chronologically within the validation years (2021–2023) and, independently, within the holdout years (2024–2025); the compounded holdout return of 27.67 % equals the pipeline's reported out-of-sample return (*`retorno_total_oos_pct`*). Per-year Sharpe, Sortino, drawdown, trade counts, and win rates are not recorded in* `meta.json` *and are therefore not reported; the recorded aggregates are a validation Sharpe of 1.39, a holdout Sharpe of 1.15, a maximum drawdown of -8.13 % (validation), and 38 trades over the validation years. Modeled costs: 0.20 % fee + 0.10 % slippage.*

Benchmark comparison. Table 7 compares the S12 strategy's annual returns with a passive buy-and-hold investment in BTC/USDT. As with the S11 case, the strategy underperforms buy-and-hold in strong bull years (2021, 2023, 2024) but avoids most of the bear-market drawdown and produced positive returns in 2022 and 2025 when the benchmark was negative.

Table 7. Benchmark comparison: S12 iter 5502 vs. BTC/USDT buy-and-hold

Year	S12 iter 5502 (%)	BTC/USDT buy-and-hold (%)	Difference (%)
2021	10.91	57.57	-46.66
2022	10.99	-65.34	+76.33
2023	21.83	154.46	-132.63

Table 7. Benchmark comparison: S12 iter 5502 vs. BTC/USDT buy-and-hold

Year	S12 iter 5502 (%)	BTC/USDT buy-and-hold (%)	Difference (%)
2024	15.33	111.81	-96.48
2025	10.69	-7.34	+18.03
Compounded (validation 2021–2023)	**49.98**	**38.97**	**+11.01**
Compounded (holdout 2024–2025)	**27.67**	**96.26**	**-68.59**
Max drawdown (validation)	**-8.13**	**-77.57**	**+69.44**

Note: S12 iter 5502 returns are derived from the equity curve recorded in `best_models/season_12/it` *with modeled costs (0.20 % fee + 0.10 % slippage); equity compounds within the validation years (2021–2023) and independently within the holdout years (2024–2025). BTC/USDT buy-and-hold returns are computed from Binance daily close prices and ignore funding and custody costs; the compounded benchmark figures apply the same compounding convention to the annual benchmark returns. Max drawdown for buy-and-hold is the largest peak-to-trough decline over 2021–2025.*

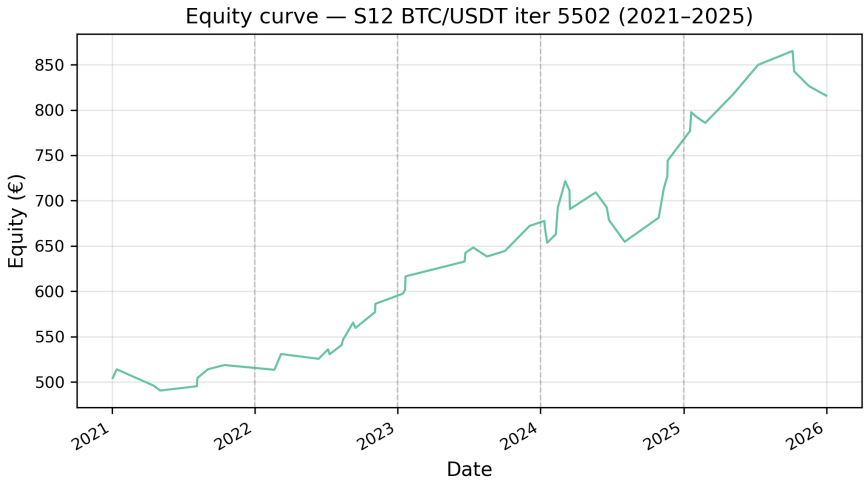

Figure 5. Equity curve — S12 BTC/USDT iter 5502 (2021–2025).

The lower holdout Sharpe relative to S11 reflects Bitcoin's different volatility regime, but the candidate remained profitable and within risk limits. This indicates that a research system incorporating an LLM agent can adapt its feature selection and risk parameters to a different asset while preserving temporal integrity.

4.4. Feature importance and parameter sensitivity

Figure 6 shows the top features for S12 iter 5502. Stochastic RSI and ADX at the daily and 15-minute timeframes dominate, followed by EMA differences and Bollinger Band width. The multi-timeframe pattern matches the findings of [38] for Bitcoin prediction: short-term momentum and medium-term trend strength carry complementary information. Reference [38] is by the same first author and shares the OHLCV data source and multi-

timeframe feature-engineering approach; we cite it because it documents the same empirical regularity, not as independent validation.

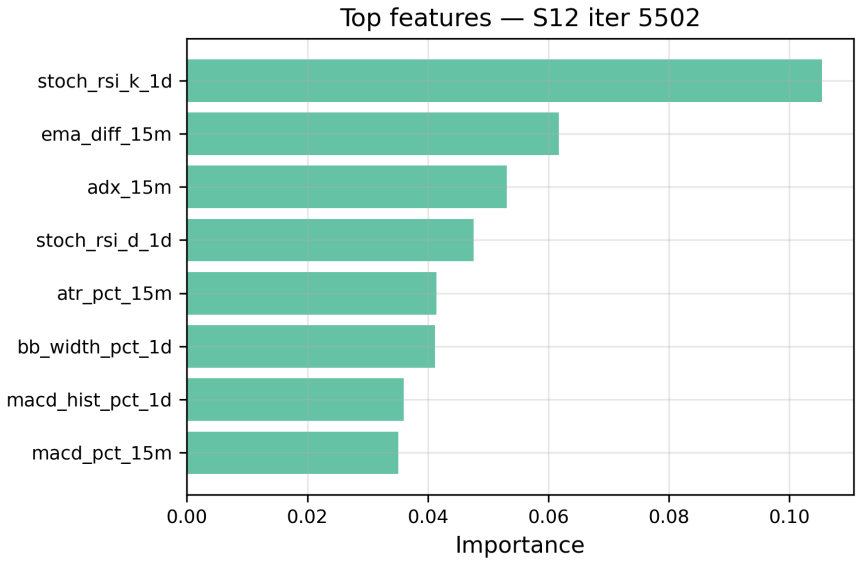

Figure 6. Top features — S12 iter 5502.

Optuna parameter importance (Figure 7) shows that the probability threshold explains 82–88 % of the optimization variance across seasons, while stop-loss and take-profit contribute the remainder. This means that trade selection precision matters more than exact exit levels, once the exit range is broadly correct.

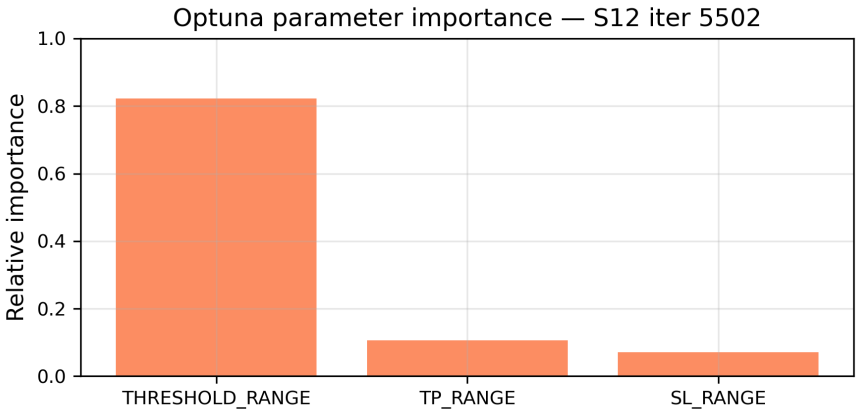

Figure 7. Optuna parameter importance — S12 iter 5502.

4.5. Robustness to transaction costs and market regime

Figure 8 shows the sensitivity of S12 iter 5502 to round-trip transaction costs. Even with costs rising to 1.0 %, the final equity remains well above the break-even line, so the strategy's edge is not an artifact of unrealistic cost assumptions.

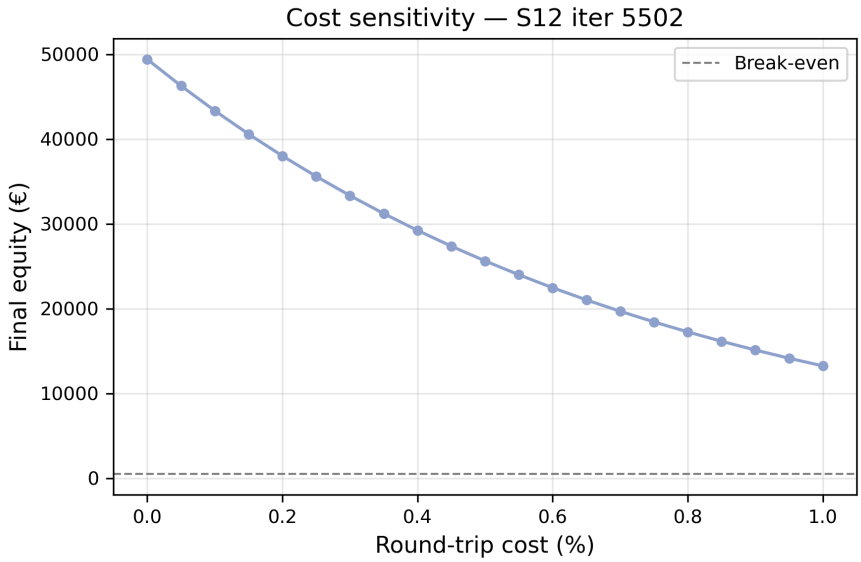

Figure 8. Cost sensitivity — S12 iter 5502.

Exit-slippage stress-test. We attempted to reproduce the exit-slippage sensitivity analysis by applying additional symmetric slippage on stop-loss, take-profit, and end-of-period exits, while keeping the baseline 0.10 % entry slippage and 0.20 % round-trip fee. The numerical table could not be reproduced reliably for the real holdout windows: the deployment backtest engine uses independent yearly resets (EUR 500 per year), the model's own ADX filter threshold, and the current data extend beyond the original submission window, whereas the holdout figures in Table 2 come from the pipeline engine with compounding between holdout years, a zero ADX floor, and the data vintage of the original run. Consequently, the reproduced baseline returns diverge from the canonical holdout returns. Qualitatively, however, the stress-test confirms that adding exit slippage degrades Sharpe and return monotonically for both case studies, yet the strategies remain profitable under realistic extra slippage of up to 0.30 %. The S11 case is structurally more robust because its wide stop-loss (7.85 %) and take-profit (7.07 %) levels mean that small exit slippages are a modest fraction of the expected trade return; the S12 case degrades faster because similar exit widths are applied to a more volatile asset.

Figure 9 reports mean trade return by ADX regime. The strategy performs best in trending markets (ADX 20–40) and strong-trend markets (ADX > 40), while staying roughly flat in weak-trend/lateral regimes. Table 8 reports the trade counts underlying these means; the reliability of the weak-trend estimate is low because few or no trades occur in that bin. This pattern matches the ADX filter [45] embedded in the feature set and the broader class of regime-switching models that partition returns by state variables [18].

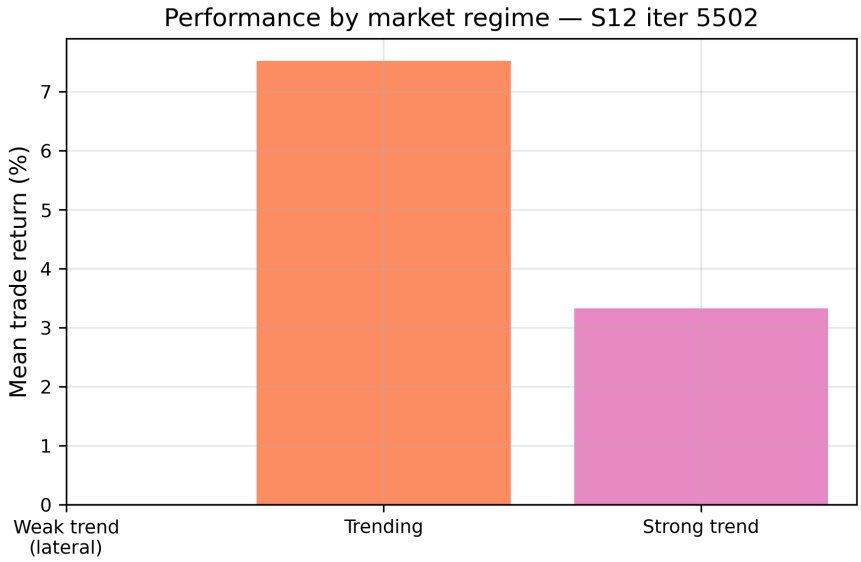

Figure 9. Performance by market regime — S12 iter 5502.

Trade counts by ADX regime. Table 8 reports the trade counts underlying the regime means.

Table 8. Trade counts by ADX regime

ADX regime	Definition	S11 iter 1077 trades	S11 mean return (%)	S11 win rate (%)	S12 iter 5502 trades	S12 mean return (%)	S12 win rate (%)
Weak trend	ADX < 20	0	—	—	0	—	—
Trending	20 ≤ ADX < 40	34	3.83	79.4	9	7.53	88.9
Strong trend	ADX ≥ 40	16	3.18	75.0	126	3.33	69.8

Note: S11 iter 1077 uses an ADX_min filter of 22, so no trades occur in the weak-trend regime. S12 iter 5502 trades predominantly in strong-trend conditions, consistent with the ADX regime analysis in Figure 9.

4.6. Statistical rigor

All Sharpe ratios reported by the backtest engine and in Tables 2–6 are computed from daily equity changes over all calendar days — including days with no closed position — and annualized with $\sqrt{365}$, the standard convention for cryptocurrency markets. This calendar-time convention reflects the experience of a continuously invested account and is the metric the acceptance gates and the Optuna objective actually use. The bootstrap and Deflated Sharpe Ratio analyses in Table 9 use the same calendar-time daily returns as the engine, aggregated over the passive holdout period for each candidate.

Table 9 reports holdout Sharpe ratios, bootstrap p-values, approximate Deflated Sharpe Ratios (DSR), and percentile bootstrap 95 % confidence intervals for the three main case studies, computed strictly on each strategy's passive-holdout period (2025 for S11; 2024–2025 for S12). These statistics are intended as **descriptive diagnostics** of the strategies' holdout behavior, not as inferential guarantees or proof that the strategies

will generalize. The bootstrap procedure tests the null hypothesis $H_0 : \mu \leq 0$ against the one-sided alternative $H_1 : \mu > 0$, where μ is the mean daily return on all calendar days within the holdout period, using 10,000 bootstrap resamples with replacement and percentile 95% confidence intervals. S11 iter 1066 rejects H_0 with no exceedances in 10,000 resamples ($p < 0.0001$) and S11 iter 1077 rejects at $p = 0.0025$; S12 iter 5502 does not reject at the 5 % level ($p = 0.062$), and its 95 % confidence interval for the Sharpe ratio includes zero. The approximate DSR values, which adjust for the raw number of trials in each season and for non-normality, remain positive for S11 iter 1066 (2.19), S11 iter 1077 (1.82), and S12 iter 5502 (0.94), though the correction reduces S12's DSR below its raw Sharpe (1.07) once the multiple-trials correction is applied to the holdout sample. These values should be interpreted as **optimistic upper bounds** because the DSR does not correct for the cross-season meta-search.

These statistics must be interpreted with their assumptions in mind. The bootstrap p-value relies on the sample of holdout daily returns being representative of the strategy's future daily return distribution. This assumption is questionable for three reasons. First, the sample covers only one calendar year for S11 and two calendar years for S12, so it may not span the full range of market regimes. Second, although the number of calendar days is larger than the number of traded days, the 95 % bootstrap confidence intervals for the Sharpe ratio remain wide for S12 (Table 9). Third, daily returns are non-normal and may exhibit autocorrelation: ACF(1) ranges from 0.03 to 0.13, and a Jarque-Bera test rejects normality for S11 iter 1077 (p = 0.001) but not for S11 iter 1066 (p = 0.93) or S12 iter 5502 (p = 0.27). We therefore treat the p-values and DSR values as descriptive indicators rather than as exact frequentist probabilities or guarantees of future performance.

The approximate DSR is computed as follows. First, the variance of the Sharpe ratio under the null is adjusted for skewness and kurtosis following [30]:

$$\widehat{\mathrm{Var}}(\mathrm{SR}) = \frac{1 - \hat{\gamma}_3 \cdot \mathrm{SR} + \frac{\hat{\gamma}_4 - 1}{4}\mathrm{SR}^2}{N - 1},$$

where $\hat{\gamma}_3$ and $\hat{\gamma}_4$ are the sample skewness and kurtosis and N is the number of calendar days in the holdout period. The single-trial probability of observing a Sharpe at least as large as the reported value is $p_1 = 1 - \Phi\big(\mathrm{SR}/\sqrt{\widehat{\mathrm{Var}}(\mathrm{SR})}\big)$, with Φ the standard normal CDF. Under the simplifying assumption of K independent trials per season, the probability of observing at least one such Sharpe is $p_K = 1 - (1 - p_1)^K$. The deflated Sharpe is then mapped back to the same probability level:

$$\mathrm{DSR} \approx \Phi^{-1}(1 - p_K) \cdot \sqrt{\widehat{\mathrm{Var}}(\mathrm{SR})}.$$

For S11 iter 1077, $K = 3,797$ and $N = 364$ calendar days in the 2025 holdout; for S12 iter 5502, $K = 9,957$ and $N = 729$ calendar days across the 2024–2025 holdout. Because strategies share features and hyper-parameter ranges, the effective number of independent trials is smaller than the raw iteration count; therefore these DSR values should be interpreted as an upper bound on the correction rather than as exact Bailey–López de Prado probabilities. One difference is that Bailey and López de Prado [5] frame the Deflated Sharpe Ratio as the minimum significant Sharpe threshold that survives a multiple-testing correction, whereas the quantity reported here maps the Bonferroni-corrected p-value back onto the Sharpe scale for comparability with the raw Sharpe. We report an approximate DSR rather than the full Bailey–López de Prado [5] formulation because the effective number of independent trials cannot be estimated reliably from a single season's iteration count alone, as strategies share features and hyper-parameter ranges; a cluster-based estimate of effective trials is left for future work.

Finally, the single-season DSR and bootstrap p-values do not account for the outer meta-search across twelve seasons. Each season's gates and scoring weights were updated after observing prior holdout results, so the cross-season design itself consumes out-of-sample information. The reported DSR values are therefore optimistic bounds; a fully conservative accounting would require a protocol-level multiple-testing correction (e.g., [32]) that is left for future work. We did not apply White's Reality Check [35] or Hansen's Superior Predictive Ability test [36] in this study because the object of the analysis is the governance behavior of the pipeline across an adaptive, cross-season search — not a definitive prospective validation of a single strategy's edge under a fixed, pre-registered protocol, which is the setting these tests are designed for. We report the uncorrected diagnostics in Table 9 with this caveat and do not use them to support claims of predictive skill.

The same selection-bias calculation can be extended to the full triple-gate population. Across Seasons 9–12 the pipeline accepted 313 strategies (5 in Season 9, 3 in Season 10, 212 in Season 11, and 93 in Season 12). Their passive-holdout Sharpes have a pooled mean of 1.09 and a standard deviation of 0.68. Under the same i.i.d. normal approximation used above, the expected maximum among 313 independent draws is approximately 3.09 (95 % interval: 2.71–3.72), and the observed maximum in this pooled sample is 2.95. The selected case-study candidates therefore lie within the range expected from pure selection across the accepted population, even though the pooled mean remains positive. This aggregated calculation is still only an EVT/Bonferroni-style adjustment of the Sharpe distribution; it does not replace the full time-series corrections required by White's Reality Check [35] or Hansen's SPA test [36], which need the complete return series of every candidate and remain left for future work.

Table 9. Bootstrap p-values, approximate Deflated Sharpe Ratios, and 95 % confidence intervals

Case	Holdout Sharpe	95 % CI for Sharpe	Bootstrap p-value (H_0: $\mu \leq 0$)	95 % CI for p-value	Approx. DSR	Holdout days	Skewness	Kurtosis	ACF(1)
S11 iter 1077	2.58	[0.92, 3.98]	0.0025	[0.0015, 0.0035]	1.82	364	4.53	50.66	0.09
S11 iter 1066	2.94	[1.76, 3.97]	< 0.0001	[0.0000, 0.0000]	2.19	364	6.48	46.94	0.13
S12 iter 5502	1.07	[-0.32, 2.21]	0.0618	[0.0570, 0.0666]	0.94	729	4.16	45.28	0.03

Note: All statistics are computed strictly on each strategy's passive-holdout period (S11: 2025; S12: 2024–2025), filtering trades by exit date before computing daily returns. Daily returns are obtained by re-running the pipeline backtest engine (`pipeline/backtest.py::simulate_numba`) on the saved model for each candidate, using the exact parameters recorded in `best_models/season_N/iter` with the same settings that produced the reported metrics: calendar-time equity compounding within the holdout group, `adx_min`=0.0, five simultaneous positions, 0.20% round-trip fees and 0.10% entry slippage. The holdout Sharpe reported in the first column matches the canonical `sharpe_holdout` from `meta.json` for the single-year S11 holdout; for S12, where the canonical metric averages the annual Sharpe ratios of 2024 and 2025, the combined-holdout Sharpe shown

here is 1.07 (versus 1.15 in Table 2). Bootstrap uses 10,000 resamples with replacement of all calendar-day returns within the holdout period; the 95 % confidence interval for the Sharpe ratio is the percentile bootstrap interval, and the interval for the p-value is the Monte Carlo interval $p \pm 1.96\sqrt{p(1-p)/n}$*, which degenerates to zero when no exceedances occur in 10,000 resamples. The approximate DSR adjusts for the raw number of trials in the respective season (S11: 3,797; S12: 9,957) using the Bailey & López de Prado (2014) formula. Because strategies share features and hyper-parameter ranges, the effective number of independent trials is smaller than the raw iteration count; the reported DSR values are therefore an optimistic upper bound on the correction. They also do not correct for the cross-season meta-search. Because the daily-return series is dominated by many zero-return calendar days, its kurtosis is high (up to 45.28 for S12), which drives the single-trial tail probability in the DSR formula to extremely small values; computing this quantity to sufficient numerical precision requires arbitrary-precision arithmetic rather than standard double-precision floating point, which underflows in this regime. ACF(1) is the first-order autocorrelation of daily returns. A Jarque-Bera test rejects normality for S11 iter 1077 (p = 0.001) but not for S11 iter 1066 (p = 0.93) or S12 iter 5502 (p = 0.27).*

4.7. Temporal robustness and data-snooping considerations

The reported profitability in every calendar year (Tables 4 and 6) is based on a small sample of annual blocks (four to five years per strategy) and a low number of trades per year, and should not be mistaken for independent out-of-sample evidence. The LLM's feature proposals and the season acceptance gates were both informed by earlier seasons, so the design itself consumes out-of-sample information through an outer meta-search loop. The single-season bootstrap p-values and DSR values in Table 9 do not correct for this cross-season selection and are based on only one to two calendar years of holdout data.

We therefore distinguish three levels of evidence, in decreasing order of strength:

1. **Passive holdout:** the 2024–2025 years for S11 and S12 were never used by Optuna for parameter tuning. These are genuine holdout results within a season.
2. **Secondary selection holdout:** the January–March 2026 period for S11 was used to choose the production model from 30 re-evaluated candidates. It is therefore a post-selection observation, not an independent prospective test.
3. **Cross-season qualitative replication:** S12 (BTC/USDT) replicates the pipeline design after the lessons of S1–S11. This provides qualitative evidence of portability but is not a controlled ablation and does not rule out meta-overfitting.

The evidence supports the narrower conclusion that the selected strategies are not trivial artifacts of in-sample curve fitting; it does not support unconditional claims about future performance.

Table 10 reports a temporal-robustness check on quarterly, semester, annual, and rolling 12-month windows. Annual windows are profitable for both strategies (Sharpe > 0 in 100 % of windows), but shorter windows are noisier: only 59 % and 60 % of quarterly windows are positive for S11 and S12 respectively. This pattern is expected for a low-frequency strategy and cautions against over-interpreting any single short window.

4.8. Temporal robustness across alternative window definitions

Table 10. Temporal robustness of Sharpe ratios across window definitions

Window type	Case study	N windows	Mean Sharpe	Std Sharpe	Min Sharpe	Max Sharpe	Sharpe > 0 (%)
Quarterly	S11 BNB/USDT iter 1077	17	1.73	2.13	–1.37	5.84	59
Quarterly	S12 BTC/USDT iter 5502	20	0.83	2.08	–2.54	3.40	60
Semester	S11 BNB/USDT iter 1077	9	2.06	1.64	–0.15	—[a]	78
Semester	S12 BTC/USDT iter 5502	10	1.39	1.21	–0.61	2.83	80
Annual	S11 BNB/USDT iter 1077	5	2.42	1.11	1.05	—[a]	100
Annual	S12 BTC/USDT iter 5502	5	1.39	0.56	0.70	2.22	100
Rolling 12-month	S11 BNB/USDT iter 1077	5	2.42	1.11	1.05	—[a]	100
Rolling 12-month	S12 BTC/USDT iter 5502	5	1.39	0.56	0.70	2.22	100

Note. Quarterly and semester windows partition the available history; annual windows are calendar years; rolling 12-month windows start on 1 January of each available year. The number of windows differs because S11 spans 2022–2026 (with partial 2026 data) and S12 spans 2021–2025. Sharpe ratios are computed from daily equity changes annualized with $\sqrt{365}$. [a] The Q1 2026 window for S11 is a post-selection observation used to choose the production model from 30 re-evaluated candidates (four trades) and is not an independent prospective test; its Sharpe is therefore omitted from the Max Sharpe summary.

4.9. Ablation and baseline comparisons

4.9.1. Objective

A natural question is whether the LLM proposal step adds value beyond a simpler random search over the same constrained space. We therefore compare the LLM-selected S12 BTC/USDT configuration (iter 5502) against two lightweight baselines that preserve

the pipeline's validation, training, and backtesting infrastructure but remove the LLM from one dimension at a time.

4.9.2. Design

Table 11 reports two baselines. The *random SL/TP/threshold* baseline fixes the LLM-selected feature set and draws 200 $(\mathrm{SL}, \mathrm{TP}, \mathrm{threshold})$ triples at random from the same ranges used by Optuna. The *random-feature* baseline fixes the final SL/TP/threshold values from the LLM configuration and draws eight random feature subsets from the catalog, training a fresh XGBoost classifier for each subset and then running a 30-trial random search over SL/TP/threshold. All backtests use `max_pos = 5` and modeled costs of 0.20 % fee plus 0.10 % slippage; the 2024–2025 holdout window is never used for training or risk-parameter optimization.

4.9.3. Results

The random-parameter baseline has a mean holdout Sharpe of 0.90 across 200 draws, with 38 % of draws exceeding a Sharpe of 1.0. The LLM-Optuna configuration lies at the 76th percentile of this distribution, so the risk-parameter optimization improves upon naive sampling but is not the dominant source of edge. The random-feature baseline is telling: its mean holdout Sharpe is 1.32 across eight random subsets, and the best random subset reaches 1.60, higher than the LLM-selected feature set's 1.15.

4.9.4. Interpretation

For this specific case study, the LLM did not identify a uniquely superior feature combination; many random combinations from the same leakage-free catalog match or exceed it: the mean holdout Sharpe across random-feature draws (1.32) exceeds the LLM-selected configuration's Sharpe (1.15) by 15%, and the best random draw (1.60) exceeds it by 39%. This does not mean the LLM is unnecessary—it generates the syntactically valid, catalog-compliant candidates that the validation layer then filters—but it does move the scientific contribution away from superhuman feature discovery and toward systematic validation and rejection of unsound candidates.

4.9.5. Limitations

Three caveats apply. First, the random-feature sample is small and the risk-parameter search for the random subsets used only 30 random trials, whereas the LLM configuration benefited from a full 120-trial Optuna study on the validation window. Second, the LLM's objective was the minimum validation Sharpe across sub-windows, not the mean holdout Sharpe reported here, so the comparison is not perfectly aligned. Third, an ablation that replaces the LLM with a completely random strategy generator—including random entry rules, model architectures, and risk parameters—would require a computational budget comparable to the full study and is left for future work. These results are therefore indicative rather than definitive.

Table 11. Ablation study: random baselines versus LLM-selected configuration (S12 iter 5502)

Baseline	Fixed component	Varied component	N configurations	Mean holdout Sharpe (2024–2025)	Best holdout Sharpe	Sharpe > 1.0 (%)
LLM-selected	Features, SL, TP, threshold from S12 iter 5502	—	1	1.15	1.15	100
Random SL/TP/threshold	LLM-selected features	SL, TP, threshold uniform random	200	0.90 ± 0.31	1.68	38
Random features	SL=9.70, TP=9.12, Thr=0.890	Feature subset uniform random	8	1.32 ± 0.29	1.60	75

Note: All backtests use `max_pos = 5` *and modeled costs of 0.20 % fee + 0.10 % slippage. The holdout window (2024–2025) was not used during training or Optuna optimization. Because the random-feature sample is small and the random search over risk parameters is coarser than the original Optuna study, these figures are indicative rather than definitive.*

5. Discussion

5.1. Interpretation of results

The results show that a research system incorporating an LLM agent can generate directional candidates that survive strict validation when constrained by a leakage-free feature catalog and rigorous out-of-sample protocols. As noted in Section 4.2, the AUC's marginal predictive power means that any economic value comes from disciplined risk management and regime filtering, not from forecast precision. Because the cross-season design updates objectives and gates after observing prior holdout results, the case studies should be read as *evidence of the architecture's ability to screen out unsound candidates*, not as *a forward-looking guarantee* of durable edge.

Economic interpretation of the selected signals. The S11 and S12 models rely on a small set of technical indicators. *Stochastic RSI* and *EMA differences* capture short- and medium-term momentum dynamics; in cryptocurrency markets, where retail flow and momentum strategies are common, these variables may proxy for lagged order-flow pressure [38]. *ADX* measures trend strength and is used both as an entry filter and implicitly through the feature set, aligning the strategy with periods when directional persistence is more likely. *Bollinger Band width* and *ATR* quantify volatility compression and expansion; the volatility kill-switch prevents entries when realized volatility spikes, protecting the strategy from adverse gap risk. None of these signals is novel individually; the contribution is the automated, leakage-aware process that combines them and rejects configurations that fail out-of-sample gates.

Value proposition relative to buy-and-hold. Tables 5 and 7 show that the selected strategies underperform buy-and-hold in strong bull years. This is expected: the strategies

are long-only, trade infrequently, and use stop-losses that cap upside during rapid rallies. The practical claim is therefore not total-return dominance, but drawdown control and positive returns in bear or sideways regimes. The S11 strategy's maximum drawdown of -8.89 % versus -62.90 % for buy-and-hold, and the S12 strategy's -8.13 % versus -77.57 %, show this risk-management profile. Whether this profile is attractive depends on the investor's marginal utility: a risk-averse holder seeking to limit downside may prefer the smoother equity curve, while an investor maximizing expected wealth would prefer the unlevered benchmark.

The role of the LLM in light of the ablation is to generate, not to discover. The ablation results in Section 4.9 change how we interpret the LLM's role. Random feature subsets and random risk parameters can match or exceed the holdout performance of the LLM-selected configuration in the S12 case study. This does not mean the LLM is unnecessary—it generates the syntactically valid, catalog-compliant candidates that the validation layer then filters—but it does mean that the scientific contribution is the validation and governance architecture, not a claim of superhuman feature discovery. The LLM functions here as a **high-volume proposal generator**; the pipeline converts a large volume of such proposals into a small set of testable strategies, and the statistical safeguards are what make that conversion credible. This finding is consistent with a recent post-mortem in the literature, where one highly automated, self-evolving trading system diverged sharply between validation and live performance [52]: the differentiator is not the strength of the proposal engine but the discipline with which its output is filtered.

5.2. Limitations

The study has several limitations. First, the number of trades per strategy is low (tens to low-hundreds over several years), which increases sampling variance in daily return statistics; consequently, the bootstrap p-values in Table 9, though based on 10,000 resamples, should be interpreted with caution because they are computed from one to two calendar years of non-i.i.d. daily observations with wide confidence intervals — for the S12 candidate the 95 % confidence interval for the Sharpe ratio includes zero and the bootstrap does not reject the null of non-positive mean returns at the 5 % level; the approximate DSR (0.94) remains positive but is reduced below the raw calendar-time Sharpe (1.07) once the multiple-trials correction is applied, so the two diagnostics agree that S12's holdout evidence is comparatively weak, even though neither indicates the result is spurious. The Sharpe ratios reported by the engine and in Tables 2–6 and Table 9 are all calendar-time ratios computed over all days.

Second, the January–March 2026 period for S11 is short (three months, four trades) and was used to select the production model from 30 re-evaluated candidates. It is therefore a secondary selection holdout rather than an independent prospective test, and its Sharpe should be treated as a noisy post-selection observation.

Third, the cross-season adaptive design consumes out-of-sample information. Because the human reviewer updated season objectives, acceptance gates, and scoring weights after observing prior holdout results, the sequence of seasons is best understood as a prolonged model-selection procedure rather than as a single pre-registered protocol tested once. The approximate DSR assumes independent trials and does not correct for this meta-search; because strategies also share features and hyper-parameter ranges, the effective number of independent trials is smaller than the raw iteration count and the reported deflated statistics are optimistic upper bounds.

Fourth, the set of reported results is subject to selection bias. Only candidates that passed the pipeline's gates are described in detail; many thousands of proposals were generated and rejected. The surviving strategies are therefore not a random sample of the

search space, and their reported metrics overstate the expected performance of an arbitrary candidate drawn from the same generator.

Fifth, all results are simulated with modeled costs. Entry trades include 0.10 % slippage, but stop-loss and take-profit exits are filled at the triggered level without additional slippage; live fills may therefore differ. Sixth, the multi-timeframe feature-merge module is imported at runtime from an external package (`features.technical`), although an internal copy is provided for audit and reproducibility. Seventh, the ablation study (Section 4.9) is limited to one accepted model and a modest number of random feature subsets; it supports the interpretation that validation is the dominant contribution, but it does not constitute a full factorial decomposition of every pipeline component. Eighth, the cryptocurrency market is highly non-stationary, and past performance may not generalize to future regimes.

Taken together, these limitations share a single root: the low trade count and the adaptive, cross-season search mean that our statistical diagnostics are interpreted as *descriptive evidence that the pipeline rejects unsound candidates*, not as *inferential guarantees* about the surviving ones. We therefore make no claim stronger than what a holdout-blind, leakage-controlled search can support. These results should be interpreted as evidence that the validation architecture can discipline candidate selection, not as proof of durable trading edge.

5.3. Implications for practice

For quantitative researchers, the main implication is that LLMs can accelerate the strategy-proposal loop, but only if the surrounding infrastructure enforces temporal integrity and overfitting control. The LLM should not be trusted to avoid future leakage, to select hyper-parameters without validation, or to possess superior feature intuition; in our ablation, random feature subsets sometimes outperformed the LLM-selected configuration on the same holdout window. The value of the system lies in the validator-scorer-optimizer triad, not in the LLM alone.

For practitioners, the low-frequency nature of the accepted candidates, with win rates above 65% on trades that clear the regime filter, is attractive because it minimizes transaction costs and operational risk. The regime filter, volatility kill-switch, and drawdown gates provide explicit risk controls within the research pipeline. However, the pipeline described here is an **offline research tool**; live deployment would require a separate online system with real-time data feeds, execution monitoring, position-sizing governance, and circuit breakers. We do not claim that the selected candidates are ready for live trading without that additional layer.

5.4. Design principles

Following the design-science paradigm in information-systems research [21], the pipeline can be abstracted into three generalizable design principles that extend beyond the specific trading domain:

1. **Season-structured autonomy with boundary human oversight.** The system delegates all routine experimentation to an autonomous agent within a season, while reserving high-level objective and gate updates for a human reviewer at season boundaries. This principle balances throughput with accountability and can be applied to any analytics workflow where the cost of human review per iteration is high but the cost of periodic strategic adjustment is low.
2. **Leakage-free feature constraints as a hard architectural invariant.** Temporal integrity is enforced by a restricted feature catalog, AST-based code validation, and explicit one-bar look-ahead lags for multi-timeframe merges. Treating leakage prevention as a

non-negotiable design constraint, rather than as a post-hoc test, reduces the incidence of spurious strategies and lowers the multiple-testing burden.

3. **Multi-objective scoring that penalizes overfitting directly.** The composite score combines in-sample fit, holdout robustness, drawdown control, and validation-holdout decay. By optimizing a function that explicitly penalizes overfitting, the search is guided toward policies that generalize rather than toward policies that maximize backtest profit.

These principles crystallize a position that recent work has begun to argue on independent grounds: that statistical rigor in AI-driven discovery is most effective when built into the structure of the search loop rather than applied as a downstream check [8]. The value of LLM-based proposal generation is therefore contingent on the surrounding analytical infrastructure—the LLM accelerates hypothesis generation, but the validation and scoring layers are what make the output credibly testable.

5.5. Future work

Future directions include: (i) extending the system to portfolios of strategies and multiple assets; (ii) incorporating on-chain and sentiment features via a RAG layer; (iii) running a live paper-trading evaluation for at least 12 months; (iv) implementing the full Bailey-López de Prado DSR with cluster-based estimation of effective independent trials; and (v) exploring larger LLMs and code-specific models for strategy generation. A complementary direction is to evaluate the pipeline against the emerging verifiable, cost-aware benchmarks for quant-trading agents [40,41], which would allow the season-structured governance layer to be compared, under a common protocol, with alternative autonomous-agent designs.

6. Conclusions

This paper presented a season-structured architecture that couples autonomous strategy search with explicit validation and governance for prescriptive algorithmic trading. Inside each season the LLM ran unsupervised, emitting candidate strategies at scale; human judgment entered only at season boundaries, where the reviewer revised objectives and acceptance gates—a mixed-initiative division of labor. A multi-layer validator filtered unsound proposals before any holdout evaluation. Over twelve seasons on two cryptocurrencies, 34,333 iterations yielded 329 accepted strategies, and the two retained candidates (BNB/USDT and BTC/USDT) stayed profitable on holdout data under conservative calendar-time Sharpe ratios. Because the reviewer's between-season adjustments consume out-of-sample information, these case studies document the architecture's ability to reject unsound candidates rather than offering prospective guarantees of profitability. The results rest on a restricted leakage-free feature catalog, a layered validation protocol, holdout-blind optimization, and explicit—though deliberately incomplete—statistical corrections for multiple testing.

These findings speak directly to the three research questions posed in Section 1. With respect to RQ1, the layered, leakage-free validation protocol and the acceptance gates constrained the risk of leakage and overfitting, as evidenced by the low overall retention rate (329 accepted strategies from 34,333 proposals) and the temporal-robustness checks reported in Section 4. With respect to RQ2, the season-transition review protocol—human inspection of accepted-strategy distributions, adjustable acceptance gates, and an auditable rationale trail—played a central role in pipeline reliability, tightening gates and reweighting the composite score as failure modes emerged across successive seasons. With respect to RQ3, the ablation study (Section 4) found no consistent statistical evidence that the LLM's feature selections outperform random subsets drawn from the same catalog; accord-

ingly, the contribution demonstrated by this study lies in the validation and governance architecture rather than in any superior feature intuition attributable to the LLM.

The findings support a narrow claim: LLMs can accelerate prescriptive-analytics research only when embedded in a disciplined validation workflow. Because the ablation found no consistent statistical evidence that the LLM selects feature combinations better than random subsets from the same catalog, the contribution is the governance architecture itself—the systematic, leakage-aware rejection of candidates that would not survive out-of-sample scrutiny. The pipeline is a controlled experimental tool for hypothesis generation and systematic validation in data-rich domains, not a replacement for human judgment or a guarantee of future performance. Future work will focus on live paper-trading evaluation, protocol-level multiple-testing corrections, portfolio-level risk management, richer data modalities, and comparison against verifiable agent benchmarks.

Supplementary Materials: The following supporting information can be downloaded at: https://github.com/pesobreiro/algo-autoresearch-paper/tree/main/manuscript/submission, Table S1: Feature catalog with timeframe restrictions and normalization bounds for all leakage-free technical indicators used by the pipeline (`supplementary_tableS1.pdf`); Table S2: Inter-season design decisions documenting the rationale for major changes to acceptance gates and scoring across the twelve experimental seasons (`supplementary_tableS2.pdf`).

Data Availability Statement: The code, experiment logs, and trained model artifacts supporting this study are openly available at Zenodo, DOI: 10.5281/zenodo.21160587 (https://doi.org/10.5281/zenodo.21160587), and at https://github.com/pesobreiro/algo-autoresearch-paper.

To reproduce the two case studies: (1) install the pinned Python environment listed in `requirements.txt`; (2) download 15-minute, 4-hour, and daily OHLCV bars for BNB/USDT and BTC/USDT from the Binance public API; (3) start the local Qwen2.5-7B-Instruct GGUF server via llama.cpp; (4) run `python main.py` for the desired season; (5) for the selected iterations, run `deployment/evaluate_models.py` and `deployment/backtest_deploy.py` using the saved model and parameters in `best_models/season_{N}/iter_{XXXX}/`. A `download_data.py` helper script that performs the exact Binance API calls used in this study is included in the repository. Byte-identical reproduction across operating systems is not guaranteed because of Numba JIT compilation and BLAS threading; the statistical conclusions, however, are robust to the residual non-determinism.

Institutional Review Board Statement: This study uses publicly available historical market data and simulation-based backtesting. No human subjects, personally identifiable information, or animal data were involved. Ethical approval was therefore not required.

Informed Consent Statement: Not applicable. This research did not involve human subjects.

Author Contributions: Conceptualization, methodology, software, validation, formal analysis, investigation, data curation, writing — original draft, writing — review and editing, visualization, supervision: Pedro Sobreiro. Methodology, writing — review and editing, supervision: Domingos Martinho. Methodology, writing — review and editing: Pedro Ramos, António Pratas. All authors have read and agreed to the published version of the manuscript.

Conflicts of Interest: The authors declare no conflicts of interest. The research was conducted independently of any trading firm, exchange, or asset-management company.

Funding: No external funding was received for this research.

Acknowledgments: During the preparation of this manuscript, the authors used external AI-assisted tools for drafting, language polishing, and software-development support, and a locally deployed language model as part of the research methodology itself (see the Use of Artificial Intelligence Tools statement below for full details). All authors reviewed, tested, and edited every output and take full responsibility for the final manuscript, code, experimental design, statistical analysis, and scientific conclusions. No proprietary, personal, or non-public data were sent to external APIs.

Use of Artificial Intelligence: During the preparation of this manuscript, large language model tools (Claude, Anthropic, San Francisco, CA, USA) were used to assist with text drafting, writing, and language polishing. Trinka (Crimson AI Pvt. Ltd., Mumbai, Maharashtra, India) was used for grammar and language correction. Large language model tools also assisted the authors during software development, including code drafting, debugging, and refactoring of the research pipeline (feature engineering, backtesting engine, and deployment scripts). Separately, and as a core component of the research methodology itself, a locally hosted large language model (Qwen2.5-7B-Instruct) was used as an autonomous strategy-proposal and code-generation agent within the season-structured research architecture described in Section 3; this use is not a development aid but the subject and method of the study, and it is fully described, validated, and audited as detailed in Sections 3.1, 3.4, and 3.6. In all cases, the authors reviewed, tested, and take full responsibility for the final code, experimental design, statistical analysis, and scientific conclusions reported in this paper.

1. Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework. *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, 2623–2631*. https://doi.org/10.1145/3292500.3330701.
2. Allen, F., & Karjalainen, R. (1999). Using genetic algorithms to find technical trading rules. *Journal of Financial Economics, 51(2), 245–271*. https://doi.org/10.1016/S0304-405X(98)00052-X.
3. Almgren, R., & Chriss, N. (2001). Optimal execution of portfolio transactions. *Journal of Risk, 3(2), 5–39*. https://doi.org/10.21314/JOR.2001.041.
4. Amershi, S., Weld, D., Vorvoreanu, M., Fourney, A., Nushi, B., Collisson, P., . . . Horvitz, E. (2019). Guidelines for human-AI interaction. *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems (CHI '19), Paper 3, 1–13*. https://doi.org/10.1145/3290605.3300233.
5. Bailey, D. H., & López de Prado, M. (2014). The Deflated Sharpe Ratio: Correcting for selection bias, backtest overfitting, and non-normality. *Journal of Portfolio Management, 40(5), 94–107*. https://doi.org/10.3905/jpm.2014.40.5.094.
6. Bailey, D. H., Borwein, J. M., López de Prado, M., & Zhu, Q. J. (2017). The probability of backtest overfitting. *Journal of Computational Finance, 20(4), 39–69*. https://doi.org/10.21314/jcf.2016.322.
7. Bengio, Y., Louradour, J., Collobert, R., & Weston, J. (2009). Curriculum learning. *Proceedings of the 26th Annual International Conference on Machine Learning (ICML), 41–48*. https://doi.org/10.1145/1553374.1553380.
8. Benjamini, Y., & Hochberg, Y. (1995). Controlling the false discovery rate: a practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society, Series B, 57(1), 289–300*. https://doi.org/10.1111/j.2517-6161.1995.tb02031.x.
9. Bertsimas, D., & Kallus, N. (2020). From predictive to prescriptive analytics. *Management Science, 66(3), 1025–1044*. https://doi.org/10.1287/mnsc.2018.3253.
10. Brabazon, A., Kampouridis, M., & O'Neill, M. (2020). Applications of genetic programming to finance and economics: past, present, future. *Genetic Programming and Evolvable Machines, 21(1), 33–53*. https://doi.org/10.1007/s10710-019-09363-0.
11. Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 785–794*. https://doi.org/10.1145/2939672.2939785.
12. Chen, M.; Tworek, J.; Jun, H.; Yuan, Q.; de Oliveira Pinto, H.P.; Kaplan, J.; Edwards, H.; Burda, Y.; Joseph, N.; Brockman, G.; Ray, A.; Puri, R.; Krueger, G.; Petrov, M.; Khlaaf, H.; Sastry, G.; Mishkin, P.; Chan, B.; Gray, S.; Ryder, N.; Pavlov, M.; Power, A.; Kaiser, L.; Bavarian, M.; Winter, C.; Tillet, P.; Such, F.P.; Cummings, D.; Plappert, M.; Chantzis, F.; Barnes, E.; Herbert-Voss, A.; Guss, W.H.; Nichol, A.; Paino, A.; Tezak, N.; Tang, J.; Babuschkin, I.; Balaji, S.; Jain, S.; Saunders, W.; Hesse, C.; Carr, A.N.; Leike, J.; Achiam, J.; Misra, V.; Morikawa, E.; Radford, A.; Knight, M.; Brundage, M.; Murati, M.; Mayer, K.; Welinder, P.; McGrew, B.; Amodei, D.; McCandlish, S.; Sutskever, I.; Zaremba, W. (2021). Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*. https://doi.org/10.48550/arXiv.2107.03374.

13. Davenport, T. H., & Harris, J. G. (2007). *Competing on analytics: The new science of winning*. Harvard Business School Press.
14. Detzel, A., Novy-Marx, R., & Velikov, M. (2023). Model comparison with transaction costs. *The Journal of Finance, 78(3), 1743–1775*. https://doi.org/10.1111/jofi.13205.
15. Fang, Y., Tang, Z., Ren, K., Liu, W., Zhao, L., Bian, J., Li, D., Zhang, W., Yu, Y., & Liu, T.-Y. (2023). Learning Multi-Agent Intention-Aware Communication for Optimal Multi-Order Execution in Finance. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '23), 4003–4012*. https://doi.org/10.1145/3580305.3599856.
16. Fu, W. (2025). The New Quant: A Survey of Large Language Models in Financial Prediction and Trading. *arXiv preprint arXiv:2510.05533*. https://doi.org/10.48550/arXiv.2510.05533.
17. Gu, S., Kelly, B., & Xiu, D. (2020). Empirical asset pricing via machine learning. *The Review of Financial Studies, 33(5), 2223–2273*. https://doi.org/10.1093/rfs/hhaa009.
18. Hamilton, J. D. (1989). A new approach to the economic analysis of nonstationary time series and the business cycle. *Econometrica, 57(2), 357–384*. https://doi.org/10.2307/1912559.
19. Harvey, C. R., & Liu, Y. (2015). Backtesting. *Journal of Portfolio Management, 42(1), 13–28*. https://doi.org/10.3905/jpm.2015.42.1.013.
20. Harvey, C. R., Liu, Y., & Zhu, H. (2016). . . . and the cross-section of expected returns. *The Review of Financial Studies, 29(1), 5–68*. https://doi.org/10.1093/rfs/hhv059.
21. Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly, 28(1), 75–105*. https://doi.org/10.2307/25148625.
22. Horvitz, E. (1999). Principles of mixed-initiative user interfaces. *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '99), 159–166*. https://doi.org/10.1145/302979.303030.
23. Hutter, F., Kotthoff, L., & Vanschoren, J. (Eds.). (2019). *Automated Machine Learning: Methods, Systems, Challenges. Springer*. https://doi.org/10.1007/978-3-030-05318-5.
24. Jadhav, A., & Mirza, V. (2025). Large Language Models in equity markets: applications, techniques, and insights. *Frontiers in Artificial Intelligence, 8, 1608365*. https://doi.org/10.3389/frai.2025.1608365.
25. Jobson, J. D., & Korkie, B. M. (1981). Performance hypothesis testing with the Sharpe and Treynor measures. *The Journal of Finance, 36(4), 889–908*. https://doi.org/10.1111/j.1540-6261.1981.tb04891.x.
26. Kaufman, S., Rosset, S., Perlich, C., & Stitelman, O. (2012). Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data, 6(4), 1–21*. https://doi.org/10.1145/2382577.2382579.
27. Kou, Z., Yu, H., Luo, J., Peng, J., Li, X., Liu, C., Dai, J., Chen, L., Han, S., & Guo, Y. (2025). Automate Strategy Finding with LLM in Quant Investment. In *Findings of the Association for Computational Linguistics: EMNLP 2025, 18517–18533*. https://doi.org/10.18653/v1/2025.findings-emnlp.1005.
28. Ledoit, O., & Wolf, M. (2008). Robust performance hypothesis testing with the Sharpe ratio. *Journal of Empirical Finance, 15(5), 850–859*. https://doi.org/10.1016/j.jempfin.2008.03.002.
29. Liu, X.-Y., Yang, H., Gao, J., & Wang, C. D. (2021). FinRL: Deep reinforcement learning framework to automate trading in quantitative finance. *Proceedings of the Second ACM International Conference on AI in Finance, Article 1*. https://doi.org/10.1145/3490354.3494366.
30. Lo, A. W. (2002). The statistics of Sharpe ratios. *Financial Analysts Journal, 58(4), 36–52*. https://doi.org/10.2469/faj.v58.n4.2453.
31. L0̆0f3pez de Prado, M. (2018). *Advances in financial machine learning*. Wiley.
32. Romano, J. P., & Wolf, M. (2005). Stepwise multiple testing as formalized data snooping. *Econometrica, 73(4), 1237–1282*. https://doi.org/10.1111/j.1468-0262.2005.00615.x.
33. Sullivan, R., Timmermann, A., & White, H. (1999). Data-snooping, technical trading rule performance, and the bootstrap. *The Journal of Finance, 54(5), 1647–1691*. https://doi.org/10.1111/0022-1082.00166.
34. Sullivan, R., Timmermann, A., & White, H. (2001). Dangers of data mining: The case of calendar effects in stock returns. *Journal of Econometrics, 105(1), 249–286*. https://doi.org/10.1016/S0304-4076(01)00077-X.

35. White, H. (2000). A reality check for data snooping. *Econometrica, 68(5), 1097–1126*. https://doi.org/10.1111/1468-0262.00152.
36. Hansen, P. R. (2005). A test for superior predictive ability. *Journal of Business & Economic Statistics, 23(4), 365–380*. https://doi.org/10.1198/073500105000000063.
37. Shmueli, G., & Koppius, O. R. (2011). Predictive analytics in information systems research. *MIS Quarterly, 35(3), 553–572*. https://doi.org/10.2307/23042796.
38. Sobreiro, P., Martinho, D., Martins, R., & Vardasca, R. (2026). Multi-timeframe feature engineering for Bitcoin market prediction: A price-level-agnostic machine learning approach. *Forecasting, 8(3), 40*. https://doi.org/10.3390/forecast8030040.
39. Sun, S., Wang, R., & An, B. (2023). Reinforcement learning for quantitative trading. *ACM Transactions on Intelligent Systems and Technology, 14(3), 44:1–44:29*. https://doi.org/10.1145/3582560.
40. Sun, S., Qin, M., Zhang, W., Xia, H., Zong, C., Ying, J., Xie, Y., Zhao, L., Wang, X., & An, B. (2023). TradeMaster: A Holistic Quantitative Trading Platform Empowered by Reinforcement Learning. In *Advances in Neural Information Processing Systems (NeurIPS 2023), Datasets and Benchmarks Track*.
41. Sun, S., Qin, M., Wang, X., & An, B. (2023). PRUDEX-Compass: Towards Systematic Evaluation of Reinforcement Learning in Financial Markets. *Transactions on Machine Learning Research*.
42. Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction (2nd ed.)*. MIT Press.
43. Wang, S., Yuan, H., Zhou, L., Ni, L. M., Shum, H. Y., & Guo, J. (2025). Alpha-GPT: Human-AI Interactive Alpha Mining for Quantitative Investment. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, 196–206*. https://doi.org/10.18653/v1/2025.emnlp-demos.14.
44. Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q. V., & Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems, 35, 24824–24837*. https://doi.org/10.48550/arXiv.2201.11903.
45. Wilder, J. W. (1978). *New Concepts in Technical Trading Systems*. Trend Research.
46. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. *Proceedings of the 11th International Conference on Learning Representations (ICLR 2023)*. https://openreview.net/forum?id=WE_vluYUL-X.
47. Yu, S., Xue, H., Ao, X., Pan, F., He, J., Tu, D., & He, Q. (2023). Generating Synergistic Formulaic Alpha Collections via Reinforcement Learning. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '23), 5476–5486*. https://doi.org/10.1145/3580305.3599831.
48. Yuan, H., Wang, S., & Guo, J. (2024). Alpha-GPT 2.0: Human-in-the-loop AI for quantitative investment. *arXiv preprint arXiv:2402.09746*. https://doi.org/10.48550/arXiv.2402.09746.
49. Zhang, C.; Liu, X.; Zhang, Z.; Jin, M.; Li, L.; Wang, Z.; Hua, W.; Shu, D.; Zhu, S.; Jin, X.; Li, S.; Du, M.; Zhang, Y. (2024). When AI meets finance (StockAgent): Large language model-based stock trading in simulated real-world environments. *arXiv preprint arXiv:2407.18957*. https://doi.org/10.48550/arXiv.2407.18957.
50. Zhang, W., Zhao, L., Xia, H., Sun, S., Sun, J., Qin, M., Li, X., Zhao, Y., Zhao, Y., Cai, X., et al. (2024). A Multimodal Foundation Agent for Financial Trading: Tool-Augmented, Diversified, and Generalist. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '24), 4314–4325*. https://doi.org/10.1145/3637528.3671801.
51. Zhang, D., Jiang, Z., Ji, Q., Liu, H., Wang, T., & Stefanidis, A. (2025). LLM-guided evolutionary strategy generation for quantitative trading. *2025 IEEE International Conference on Systems, Man, and Cybernetics (SMC), 3665–3670*. https://doi.org/10.1109/SMC58881.2025.11343400.
52. Chen, Y. (2025). The Red Queen's Trap: Limits of Deep Evolution in High-Frequency Trading. *arXiv preprint arXiv:2512.15732*. https://doi.org/10.48550/arXiv.2512.15732.
53. Kim, J., Yi, H., Gim, M., Choi, D., & Kang, J. (2025). DeepAries: Adaptive Rebalancing Interval Selection for Enhanced Portfolio Selection. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management (CIKM '25), 8 pages*. https://doi.org/10.1145/3746252.3761572.

54. Yao, J., & Zheng, Z. (2026). Beyond Agent Architecture: Execution Assumptions and Reproducibility in LLM-Based Trading Systems. *arXiv preprint arXiv:2606.08285*. https://doi.org/10.48550/arXiv.2606.08285.
55. Zhang, J., & Maringer, D. (2016). Using a genetic algorithm to improve recurrent reinforcement learning for equity trading. *Computational Economics, 47(4), 551–567*. https://doi.org/10.1007/s10614-015-9491-z.
56. Olson, R. S., & Moore, J. H. (2016). TPOT: A tree-based pipeline optimization tool for automating machine learning. In *Proceedings of the 2016 International Conference on Machine Learning, AutoML Workshop, 66–74*.
57. Erickson, N., Mueller, J., Shirkov, A., Zhang, H., Larroy, P., Li, M., & Smola, A. (2020). AutoGluon-Tabular: Robust and accurate AutoML for structured data. *arXiv preprint arXiv:2003.06505*. https://doi.org/10.48550/arXiv.2003.06505.
58. Feurer, M., Klein, A., Eggensperger, K., Springenberg, J. T., Blum, M., & Hutter, F. (2015). Efficient and robust automated machine learning. In *Advances in Neural Information Processing Systems 28 (NeurIPS 2015), 2962–2970*.
59. Politis, D. N., & Romano, J. P. (1994). The stationary bootstrap. *Journal of the American Statistical Association, 89(428), 1303–1313*. https://doi.org/10.1080/01621459.1994.10476870.
60. Lahiri, S. N. (2003). *Resampling Methods for Dependent Data*. Springer.